M363 Unit 5
UNDERGRADUATE COMPUTING

Software engineering with objects

Classes and associations

Unit 5

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1611 5

1.1

CONTENTS

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Tara Marshall, Print Procurement

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

This unit will introduce sufficient UML to enable you to build simple static models. The unit covers:

- the production of a class model for a software system, including the possible classes and their relationships (Section 2);
- a way of representing constraints on a class model (Section 3);
- the role of generalisation and specialisation in a class model (Section 4);
- the constraints that can apply to model elements (Section 5).

2 Class and object models in UML

This section introduces the main modelling elements that you will find in a class model, which presents a view of the static structure of a potential software system. If you structure software around objects in the problem domain, you are more likely to design systems whose components remain useful as the problem domain evolves. In this unit, you will build a class model that is an acceptable solution to the problem posed by the requirements. A class model describes the objects and the relationships that are needed between them to implement the required functionality. You therefore need to find ways to:

- describe what the required behaviour is, using use cases and activity diagrams;
- define the classes composing the software (the subject of this unit);
- describe how instances of these classes interact to achieve the required system behaviour.

You might choose to consider what a class model should look like first of all, so that you can write use cases with a firm idea of what sort of relationships between classes are expressible in a class model. In practice, use case modelling and class modelling will 'feed off' each other. In the early stages of analysis, you may swap between the two as you begin to understand more about the domain and the problem that your project intends to solve.

Models are always built for a purpose. You cannot just examine a model and judge whether it is good or bad. You need to establish whether the model represents what is required of it. The way you model a person, for example, will depend on what you want to do with the information about the person. A medical system and a hotel system have very different concerns: the former might model blood groups; the latter might model hotel rooms. Therefore, these two systems require different models for the concept of a person. The question is only whether the models are adequate for the purpose. This is never a technical question, but depends on someone who understands the domain being modelled. This person might be a hospital administrator or a hotel manager. We will use the term **domain expert** for a person who understands the relevant part of the application domain rather than the technology used to automate it. Without a domain expert, your best chance of identifying a good concept to include in the model comes from an understanding of the problem domain gained from your discussions with the potential users.

2.1 The search for classes

The first step in building a class model is to identify objects in the problem domain. Because any objects in the system must eventually be instances of a class, they are an excellent source of candidate classes. However, identifying objects is not an easy task. A number of techniques have been suggested that address the problem, but none of them should be applied mechanically.

Which objects should you look for, and where can you find them? In *Unit 2*, we identified the term *requirements elicitation* for the source of the requirements. In effect, you would search through all the documents, forms and so on, that were supplied by the customer. The following categories are useful sources of relevant objects:

- **tangible objects** – the physical things in the domain such as rooms, bills, books and vehicles;
- **roles** – the roles played by people in the domain, such as employees, guests and members;
- **events** – the circumstances, episodes, interactions, happenings and significant incidents, such as room reservations, vehicle registrations, orders, deliveries and transactions;
- **organisational units** – the groups to which people belong, such as accounts departments, production teams and maintenance crews.

Use these four categories for guidance. They are pointers to alert you to the objects or entities that might result in appropriate concepts for your models. It is worth stressing again that there is no guidebook or recipe for producing a model, nor is there an ideal model. Object-oriented analysis does not differ in this respect from other methods for software development.

Sometimes it is hard to decide which category is the right 'home' for an object or concept. The reason behind this difficulty is the potential for different interpretations of the same concept. Which interpretation is appropriate will usually depend on the particular circumstances. Furthermore, how you model a concept and its properties is affected by what you believe that concept represents, and what its role is in the problem domain.

A very common document that needs to be created at the beginning of a project is an agreed glossary of terms, or a data dictionary. It cross-references the different words that are to be used on the project, and explains how they interrelate – particularly when it comes to your understanding of key concepts in the problem domain. For instance, what is a *sale*? Does it come into existence when hands are shaken, or when documents are signed, or when goods are delivered, or when payment is received? Where are the goods produced? Who delivers the goods? Is a payment to be made in one sum, or is it staged according to delivery? There will be many terms and phrases used in the problem domain, and the majority will have some local meaning. Without the agreement on vocabulary that a glossary gives, there is little hope of building a good object model where the meanings of the classes need to be clear.

In this course, you have seen that requirements can be expressed in the form of 'use cases'. While looking for the use cases, you could simultaneously search through the different texts to find the objects that indicate candidate classes.

A list of nouns makes an excellent starting point when considering candidate classes for a conceptual model. However, the list may contain many nouns and noun phrases that will not make good classes. At some point you need to decide which ones to keep and which to discard. We suggest that you use the following guidelines:

- the same concept is often given different names, and you should identify those which are redundant;
- the same name might be used for different concepts, and you should rename them appropriately;
- many names are not important or not independent enough to be considered as concepts on their own; some of these may be used as properties of other identified concepts, or may be used as a concept that connects two other concepts;
- some names refer to concepts that are not relevant to the situation being considered: they are outside the scope of the problem that we are trying to understand or they are part of the way that we define things (such as *system*).

Sometimes you may also find that you cannot determine exactly what a particular word or phrase means. Naturally, you should clear up such ambiguity with the users.

Some names will certainly be redundant, because most organisations use several different words for one concept. For example, the terms ‘account’, ‘posting code’ and ‘ledger’ might all turn out to have the same meaning. Often, a single noun has several distinct meanings in different parts of an organisation. In different contexts, ‘policy’ might mean a piece of paper, a legal agreement, or a person’s insurance cover. It is not your job as a modeller to tell domain experts how to describe their subject, but you must be wary of the complexities of their language.

Because you are not yet considering the behaviour of the system, but just the things involved, you may be tempted to exclude some or all of the events (and operations) that you have found. However, the presence of a name of an event, such as ‘marry’ or ‘reserve’, which are verbs, can be a useful clue to a thing about which something needs to be said. For instance, if you wish to record the date when someone marries, you might introduce a class named *Marriage* (a noun). Also, if you need to record when a room becomes reserved, rather than whether or not it exists, you might introduce a class named *Reservation*.

Another criterion excludes nouns that are outside the scope of the system. This sounds obvious, but it can often be difficult to tell. A list of candidate classes for a petrol station system is likely to include ‘customer’. At a later stage in modelling, it might become apparent that the system need not record information about the customers as individuals. The system is concerned with the fact that petrol was pumped and money was received. The unifying concept is probably something similar to a *purchase transaction*, which involves dispensing and paying for the petrol.

The easiest classes to find are usually those that represent tangible, real-world entities, such as *Book*, in a library context, and *Room*, in a hotel context. Moving slightly deeper into the system, the different roles in a domain may provide more ideas. A *Person* may at different times have the behaviour of a *Parent*, an *Employee*, a *Customer* and a *LibraryMember*, and so it may be useful to keep these role classes separate from the *Person* class.

Classes that are easy to overlook are those that represent a transaction over a long period. For example, a *MortgageApplication* might be a useful class that could represent the state of a complex interaction with a lifetime of months.

When we looked at use cases as a way of representing requirements, we used the word actor for someone or something external to the system that can initiate interactions with it. In the case of a garage, the actors probably include the *Customer*, the *Cashier* and the *Manager*. It is often not clear whether actors need to be represented in a software system. If the cashiers are merely the agents for accepting money and recording the payment of transactions, the software system may not need to model them. If you need to record information about cashiers, such as the hours they work or which transactions they handle, they must be modelled.

SAQ 1

(a) Why do object modellers concentrate on nouns?

(b) What are the main criteria for filtering a list of nouns in order to remove inappropriate ones and settle upon a more suitable set of candidate classes?

ANSWER.....

(a) The nouns represent the things in the domain being modelled, and things are more stable than actions, which are expressed as verbs.

(b) There are three basic criteria that can be applied as follows:

- redundancy;

- not important or independent enough, such as an attribute of another class rather than a class in its own right;
- lack of relevance to the problem domain; either beyond the scope of the desired system, or part of the language used for modelling.

In addition to these basic criteria, we should pay particular attention to events. Will an instance of that kind of event have state, behaviour and identity that are significant in the problem domain? A loan of a book from a friend might not be worth modelling, for example, but a bank loan is significant: it must be paid back and it attracts interest so that you pay back more than the original loan.

In all cases, you should clear up any ambiguity with a domain expert or the users themselves.

2.2 Classes and properties

A class will usually have properties that are important to the concept the class represents. For example, the name of a *Person* may be important, or you might want to associate the *Person* with the *Company* the person works for. You have worked with attributes of classes when programming in Java; attributes are examples of properties of classes that are intrinsically connected to the class, such as a person's name. Associations are examples of another kind of property, one that represents a relationship between classes. Attributes and associations will be discussed further, later in the unit.

You have seen associations in Unit 3. To reiterate, associations are relationships between two types, in this case between two classes.

2.3 Classes and objects

In object-oriented modelling, you must carefully distinguish between objects and classes. An object stands for something real in a domain expert's world, such as a specific customer or a specific room in a hotel. The attributes of an object have values that can be tested. A class describes all possible objects of that type, defining what the objects have in common. If you talk about the class *Person*, anything you say about the class is really a general statement about all possible persons, such as *jack*, *jill* and *jenny*. Attributes of a class do not have values, but instead indicate the values that instances of the class (that is, objects) may have. A class describes what is true about all objects of that class at all times.

2.4 Classes and time

Whether or not something in the world should be modelled as a distinct class may depend on what happens over time. You should consider the life histories of instances, and how their behaviour may change over time.

Life histories of instances

A class is a template for a set of objects. If you ask questions about when the objects are created and when they are removed, you may uncover complexities that you have failed to model. In a model of a hotel, you might have a class called *Guest*. When are instances of the *Guest* class created? It might seem that the hotel system should create an instance when checking in a newly arrived person. That is, a *Guest* object would not exist unless and until a guest has actually checked in. Such a model may or may not be a good representation of the world imagined by the hotel domain experts.

If the hotel accepts reservations, you may wish to record the reservation against a person who has not yet checked in. Do you now want to say that a guest represents either a person who has checked in or someone who has made a reservation but has not yet checked in? If so, you should ask whether the entity staying in a room and the entity making a reservation are members of the same class. A company can make a reservation and pay the bill, but a company cannot stay in a room. Perhaps you should restrict the word 'guest' to refer to a person who is currently staying in the hotel, and use the word 'reserver' for an entity that makes a reservation. In that case, reservations would be made by reservers, and rooms would be allocated to guests.

Similarly, you might ask when a *Guest* object should be deleted from the system. One choice is that any *Guest* who has checked out and satisfactorily paid the bill should be deleted. In some hotels this would be an adequate model, but in other cases the hotel would want to keep records relating to a guest, the room that was occupied and the dates of occupancy, in case lost property were to be found, or disputes over bills were to arise. Another complexity comes from the fact that sometimes the guest is not the entity that pays the bill. Thus, again, you might be tempted to split the notion of guest into those aspects to do with being on the premises, and those to do with contractual obligations.

Whatever the meaning of the class *Guest* is, it is vital that it be documented. Merely choosing a good English name like 'guest' to name a class is not enough. You must be precise about what set of instances it actually represents. This sort of explanation does not belong in a UML diagram but in the project glossary, which defines every term to be used. For example, you might put the following entry into the project glossary.

> *Guest*: a *Person* who is occupying or has occupied a *Room* in the *Hotel*. See also *Reserver* and *Payer*.

Change of behaviour over time

When modelling, you should assume that an object cannot change its class during its lifetime. Unless you use some tricky implementation techniques, a *Room* cannot become a *Hotel*, and a *Guest* cannot become a *Reservation*. An instance is created as an instance of a particular class, and remains an instance of that class until it is destroyed. This may seem obvious, but it is actually a very strong restriction. For example, when modelling a hospital you might conclude that you need classes called *Patient* and *Doctor*. There would be instances of both classes; doctors would treat patients, and patients would have appointments with doctors. However, if a doctor were to become ill, the doctor would need to become a patient. If you have doctors and patients using different classes, you would not be able to represent this change in status, because objects cannot change their class dynamically.

Another example is a school administration system, which might use the classes *Teacher*, *Parent* and *Child*. A person can be both a parent and a teacher, and teachers can be parents. Eventually, children grow up and might become parents, teachers or both. It may be possible to be all three! These possibilities show that you must think carefully about what real-world entity an object represents, and whether having more than one object for a single entity is a problem.

2.5 Class and object models

A **class model** indicates which classes are in the system and describes the associations between those classes. In the same way that a class describes what is true for all objects of that class, a class model describes what is true about the entire system at all times. In contrast to the class model, an **object model** is a collection of objects, where every object in the model is an instance of a class in the class model. The object model represents one specific configuration of the system – a **snapshot** of the system at one instant in time. No object can have attributes that are not in the class model, and neither can objects have relationships that are not in the class model. Just as an object is an instantiation of a class, an object model may be thought of as an instantiation of a class model.

As the software that you want to produce will be expressed in terms of classes, it is tempting to jump straight from a list of candidate classes to an attempt to build a class model. However, at this point the classes are really just a generalisation about all possible individuals. If you generalise too quickly, you can easily overlook particular cases that do not fit the generalisation. For this reason, most modellers find it useful to construct some object models that show interesting snapshots. The snapshots consist of a number of concrete instances of relevant candidate classes and the relationships between them. The modeller generalises the object models to a class model, iterating until the requirements are satisfied. All of the eventual object models must be instantiations of the eventual class model, without exception.

An object model is visually represented using an **object diagram**. A class model is visually represented using a **class diagram**.

2.6 Object diagrams

As mentioned in Subsection 2.5, an object diagram is a visual representation of an object model, showing a snapshot of the system at some point in time. Figure 1 shows an object diagram representing a snapshot of a hotel system. Each object is represented by a rectangular box. Inside each box is a name chosen for the object, followed by a colon and the name of a class of which the object is an instance. Object names, such as *theRitz : Hotel* in Figure 1, are underlined to distinguish them as individual instances (rather than classes).

We will continue to use underlining to identify objects in diagrams, but not in the text unless there is a need to identify an object in a particular context.

You can add a second compartment to each box to include the names and values of any useful attributes. In Figure 1, for example, the object *r401 : Room* has a Boolean attribute called *enSuite* set to *true*.

Lines drawn between objects are called **links**. They represent particular cases of one object 'knowing about' another object. Each link is an instance of an association, or a relationship between classes. Thus in Figure 1, the object *theRitz : Hotel* knows about three guests and three rooms. Two of the guests know about specific rooms: *jack* knows about *r123*, and *jill* knows about *r307*. At this stage, *jenny*, the third guest, does not know about a room.

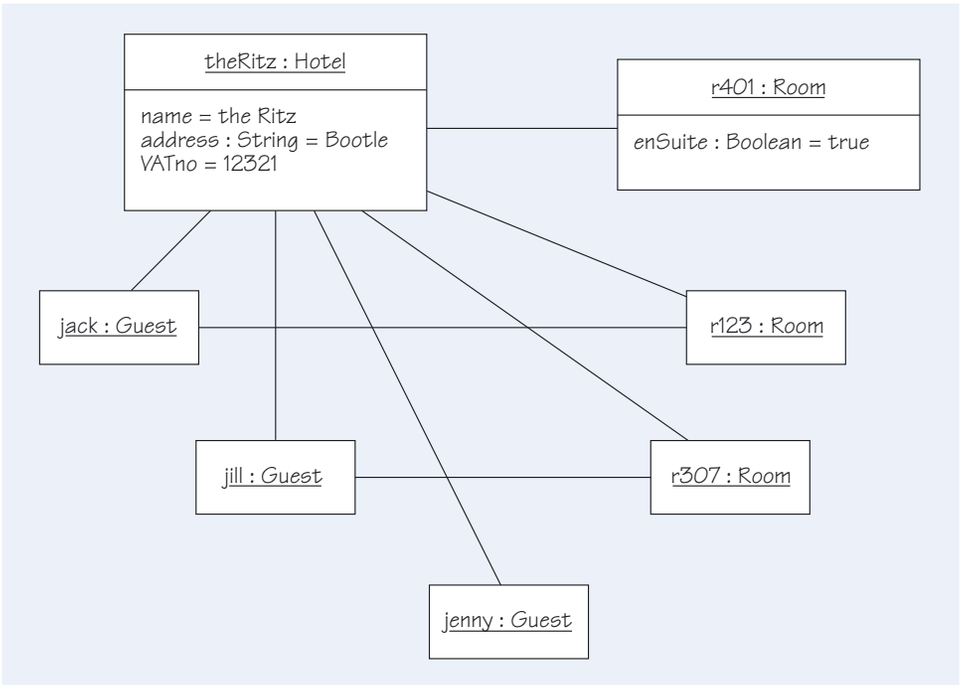

Figure 1 An object diagram

Object diagrams are extremely useful for checking your understanding of concepts and relationships. They are a fruitful source of special cases where requirements are not precisely clear and therefore require clarification from the domain expert and stakeholders. For example, you can use one or more of the object models to walk through the different scenarios for each use case.

Since an object diagram is a snapshot of the system (a representation of the system at a particular moment in time), pairs of them can be useful in showing the system before and after some operation has been performed on it. For example, in the hotel system, we can use pairs of object diagrams to show how new *Guest* objects are created, and then show what happens when a guest checks in.

For example, you can use object modelling to gain some understanding about the appropriate multiplicities for your class diagram. We will return to the subject of multiplicity in Subsection 2.10.

An object diagram can be very useful for determining whether some configuration of objects is valid. However, and as noted above, it cannot express general structural properties. In Figure 1, it is not clear from the diagram if the hotel always has exactly three guests and three rooms, or whether this is simply the situation at the moment. A class model expresses a generalisation of all possible object diagrams. A class diagram visually represents a class model.

SAQ 2

Explain why object diagrams cannot form the basis for a software specification.

ANSWER.....

Object diagrams represent particular states of the system at particular moments in time, whereas a specification must describe all valid states of the system, at all possible times.

2.7 Class diagrams

A class diagram is a visual representation of a class model. It consists of boxes representing classes, and lines representing associations. A box may contain up to three different parts, as opposed to the two parts found in an object diagram. It always contains a class name in the top section. It may have a second section containing attributes and a third section containing operations (this third section does not appear in object diagrams). A class diagram represents what all possible instances of the class have in common, rather than the particular values of any given instance. It acts as a template when creating instances, to ensure that the resultant objects all have the same structure and behaviour.

Recall that a class model expresses generalisations about all valid object models.

When you use a class diagram in a conceptual model, the main aim is to represent the information that exists in the problem domain. Hence the class diagram will show the key concepts as classes, their properties as attributes, and their relationships as associations.

Figure 2 shows a class definition and some typical instances of it. The class defines the structure and behaviour that all the instances have in common.

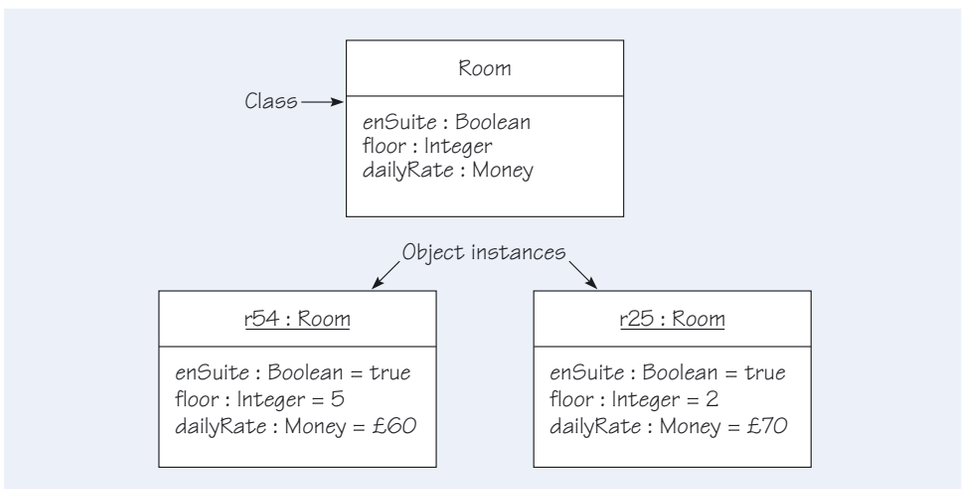

Figure 2 A class and some instances

2.8 Identity

One of the essential parts of the object-oriented way of looking at the world is the notion that every object is inherently unique and different from every other object. Every object has a unique identifier that does not depend on the current value of any of its attributes, or even on what class it is.

Consider the *Doctor* class shown in Figure 3. You can imagine numerous instances of such a class, each having its own values of the attributes. However, there is no reason why you could not have two different instances that have identical values of all attributes. This situation would arise if two cardiac surgeons named John Wilson work at the same hospital. It would also arise if, for some reason, two objects were created for the same John Wilson. Even though all the attributes are identical, from the object point of view each instance is a distinct object, which may or may not be what is desired. The modeller faces two problems: providing some means (for example, an association) of finding a desired object instance; and ensuring that enough information is included in the class to distinguish one object sufficiently from another.

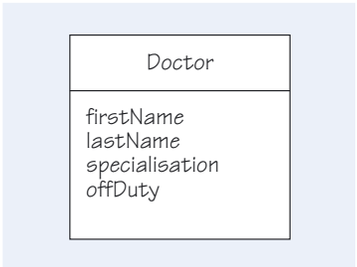

Figure 3 The *Doctor* class

As an example, consider a patient/doctor relationship, where objects of the *Patient* class are linked with objects of the *Doctor* class. If the second case above were to occur, multiple objects that stand for the same real-life doctor would be created. Imagine setting the attribute *offDuty*, of an instance of *Doctor*, to *true*. If there are multiple object instances representing the same doctor, then the value seen for *offDuty* would depend on which instance was accessed. To permit the correct *Doctor* object to be associated with the patient, both problems described in the previous paragraph must be solved. The model, or perhaps the implementation, must include some way to find all the doctor objects, and there must be enough information in the object to identify which real-life doctor the object stands for.

SAQ 3

(a) In a windowing system, a window may be converted to an icon, and back to a full window. What operations can be performed on full windows but not on iconised windows? Would a model containing the classes *Icon* and *FullWindow* be able to capture the distinction adequately?

(b) In connection with rooms, the hotel manager's vocabulary includes the words 'occupied' and 'free'. How might such words be represented in a class diagram?

(c) In your model in part (b), will your decision about occupancy change if you have to include the fact that a room must be cleaned before the next guest occupies it?

ANSWER.....

(a) Scrolling and maximising can be done on full windows but not on iconised ones. It will be difficult to model the distinction between a full window and its iconised version adequately by using two different classes, since an object (in this case, the window) cannot dynamically change its class. A solution to this problem might be to have a single class in which an attribute makes the distinction.

(b) Two ways come immediately to mind:

- as an attribute of the *Room* class;
- as an association between the *Room* and *Guest* classes.

Either is quite acceptable as a way of recording the information.

(c) No. The cleaning of a room certainly depends upon whether or not it is occupied, but not on how we choose to model occupancy. (You would include this requirement relating to cleaning in a dynamic model, such as a sequence diagram or state diagram. This will be discussed in a later unit.)

2.9 Attributes

In an object-oriented program, **attributes** of a class are represented by stored data values held in each object instance. However, during modelling, you use the word 'attribute' in a more abstract way, to refer to important properties of the class. If you are building a conceptual model of how a hotel works, the attributes of a class are just those parts of the domain expert's vocabulary that apply to that class. Thus, a hotel manager will talk about whether a room has *en suite* facilities or which floor it is on. You need a way to model these terms, and you use the middle part of the class box to record these attributes.

A *Person* might have an *age* attribute, which means that people know their own age, but this does not commit us to storing age explicitly: it might be computed from a stored date of birth.

The attributes listed in a class diagram indicate which properties of the class are important, but no values are shown. Attributes have values in objects (instances of classes), and these values are shown in object diagrams. Attributes define the state of an object at any one time. For example, if in a personnel system you need to model whether or not a person is married, there must be some attribute that differs between a married person and an unmarried one. At this stage in modelling, that is all that needs to be said.

SAQ 4

(a) Does invoking an operation on an instance of a class always change the object's state?

(b) What does an attribute of a class represent?

ANSWER.....

(a) No. Not all operations are intended to change an object's state. For example, you might provide an operation on the *Guest* class to respond with the address for any particular instance (object) of that class.

(b) An attribute represents a particular property (a named value) of the class that each instance of that class will have. Whatever else the attributes of a class are used for, at any one time they collectively define the state of an instance of the class.

2.10 Associations

We have said previously that associations are relationships between two types. More generally, we might say that an association relates two different concepts by a third. UML defines an association to be a semantic relationship between two types that connects their instances. In the case of class diagrams, the types are classes, and their instances are objects. A class model indicates what possible connections *can* exist, while an object model indicates what connections *do* exist at that instant in time.

Object models are useful for deciding what connections should be added to the class model. To see what the connections might be between sets of objects, we will examine the relationships in the hotel chain example. We can simplify the hotel chain problem, which we introduced in *Unit 4*, and identify three core concepts: hotel, room and guest. In UML, we would represent them as three classes: *Hotel*, *Room* and *Guest*. How might these three concepts be related?

In our model of the hotel reservation system, we will define a *Guest* (that is, an object of the *Guest* class) to be a person who has contacted one of the hotels in the chain to make a reservation. That is, a *Guest* is a person who is known to the hotel chain. This may mean that the person is currently staying in one of the hotels, has stayed in a hotel

previously, or has made a reservation for some time in the future. So a *Guest* is related to:

- a *Hotel* by virtue of having made a reservation;
- a *Room* by virtue of currently occupying a room or having stayed in a room previously.

In addition, a *Room* is related to a *Hotel* by being physically located in a hotel. These three associations are illustrated by the class diagram in Figure 4.

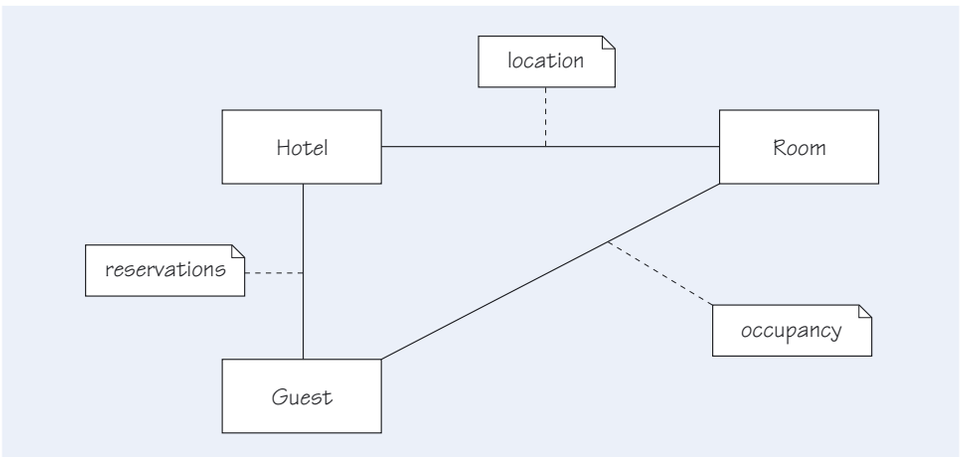

Figure 4 Some associations between *Guest*, *Hotel* and *Room*

As a temporary convenience, we have used notes to label the three associations as *reservations*, *occupancy* and *location*. Better forms of labelling will be introduced later in this unit.

Multiplicities

You should remember to check the associations in your class diagram with a domain expert. A good class diagram brings no surprises to the domain expert, because it bears a strong resemblance to the relevant part of the world that he or she is familiar with.

Once you have decided what classes to have in a model, and their interrelationships, the next most important decision is to determine how many objects are involved in a given relationship. Does our system contain a single hotel object or several? Is there a limit on the number of rooms a hotel can have? Is a guest always associated with a room, or are there guests who have several rooms or none at all?

UML provides a very rich language for modelling all such distinctions. One of the most important aspects of an association is its **multiplicity** – how many of one sort of thing can be related to another sort of thing. The annotations we will be using are shown in Figure 5.

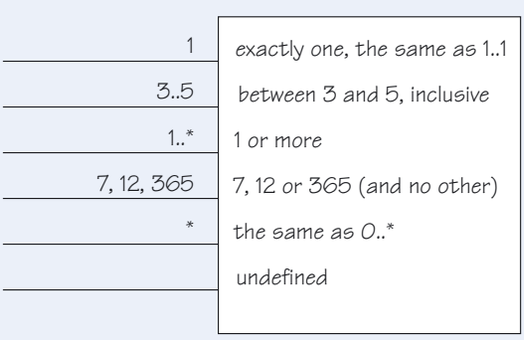

Figure 5 Annotations for associations

The general form for multiplicities uses two integers to express the range of allowable values:

minimum..maximum

The symbol * is used in place of the second integer when there is no upper limit to the allowable values. Thus the range 3..* means '3 or more'. The symbol * on its own can be used instead of 0..*, which means '0 or more' – often read as *many*. You can have several ranges in a multiplicity expression separated by commas: for example the expression 0,2..* means that there can be either no associated objects, or two or more.

In the early stages of modelling, such as producing the class diagrams for a conceptual model, you may choose not to show any multiplicities until you are confident that you have understood the problem domain.

Each end of an association has a multiplicity. By stating a multiplicity, you are saying how many instances (objects) of one class can be linked with a single instance (object) of a class at the other end of the association. Said in another way, if there were an observer standing at the object on the other end of the association and looking across the line, the multiplicity indicates the possible number of objects that the observer would be able to see. An example of the use of multiplicities is shown in Figure 6. You should consider this to be an early attempt at identifying the appropriate multiplicities for the hotel chain example (we have made some of the multiplicities deliberately unrealistic as you will see).

Figure 6 has been derived from Figure 4 by adding a fourth association to represent the history of the occupation of a room. Notice that labels have been added to the ends of some of the associations. These labels are **role names**, and are used both to name the end of an association and to indicate the purpose of the association for the class at the opposite end of the association from the label. Role names are further discussed below.

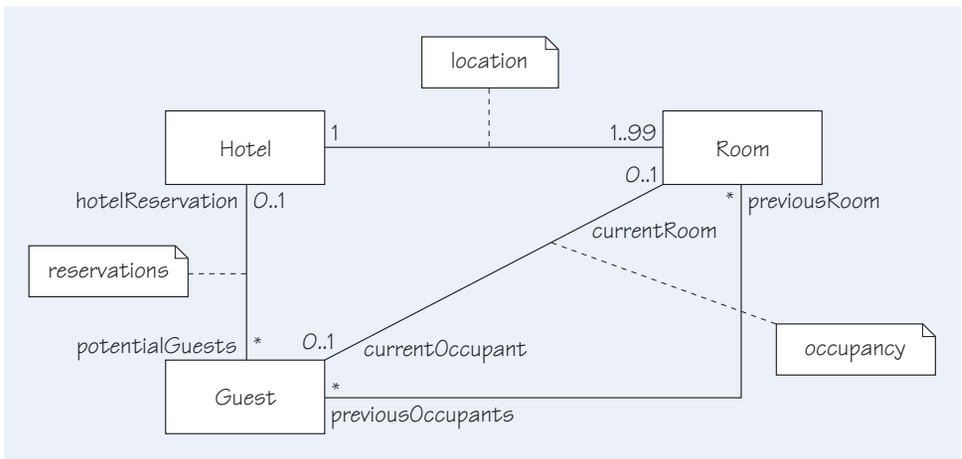

Figure 6 Examples of multiplicities

How do you interpret the multiplicities in such a diagram? It is easy to get confused by the generality, because each class represents all its instances. The best way to interpret the diagram is to think of it as saying something about each individual instance. For example, in the *location* association, the multiplicity 1..99 means that each individual hotel has at least one room and no more than 99 rooms. At its other end, the same association notes that each room is located in precisely one hotel. The *occupancy* association shows that the number of guests currently occupying each room is either 0 or 1; that is, either the room is unoccupied or there is exactly one person in it (not perhaps a realistic statement, and one that should be checked with a domain expert).

See Exercise 2 for a further discussion of the multiplicities and role names in Figure 6.

One way of arriving at the hotel model shown in Figure 6 is to draw a series of object diagrams in which you explore the potential links between pairs of objects. You need to

take a number of snapshots before you can make a generalisation about the appropriate multiplicities for a class diagram. However, you are not drawing object diagrams for their own sake, but instead to learn something about the domain. Be wary of overdoing it, and stay focused on the reason for all those snapshots.

The hotel example in Figure 6 is still a greatly simplified model, used to illustrate UML notation. For example, it allows no more than one guest per room. In addition, instances of *Guest* can have no more than one reservation at a time. What happens when the person, represented by that instance of *Guest*, wants to reserve a room at more than one hotel in the chain? Depending on the context, you might choose to change the multiplicity at the *Hotel* end of the *reservations* association from `0..1` to `0..*`, although you would need to resolve the problem of overlapping reservations.

Association names and role names

As you saw in the discussion of the abstract noun system earlier in this course, the connections between things are as important as the things themselves.

A model containing only classes (things) does not give you a working system. You need the associations (connections) that provide the 'glue' to hold it together. As associations are such important parts of models, you will frequently need to discuss them. We need to be able to name the associations in some way. In Figure 4, we named the association with a comment, which is not the best way to name associations. UML provides two mechanisms for naming associations: placing an association name in the middle of the association line; and using role names.

Association names are usually verbs. They represent the use that objects at one end of the association make of the objects at the other end of the association. A potential difficulty with a verb is that one does not know which way to read it. Does an *Employee* work for a *Company*? Or does a *Company* work for an *Employee*? Both make sense in different models. UML allows a little black triangular arrowhead to be added to the association name to indicate which way to read it. Figure 7 illustrates a named association. The arrowhead shows which direction of reading makes sense of the association name, but it does not indicate that the association is unidirectional.

Figure 7 Showing the direction in which to read a named association

Often you need to be able to read the association in both directions, so it would be very nice to have two different association names. Unfortunately, UML only allows one name per association. Role names allow us to overcome this restriction.

You can easily imagine how these role names might become the names of variables in an implementation model.

Role names allow you to name the ends of associations. The name you give to one end of an association depends on the purpose, or role, of the association from the point of view of the object at the other end. Consider an association between a *Room* and a *Guest*. There are two roles: the role the guest plays for the room, and the role the room plays for the guest. As roles are from the point of view of an object on one end of an association, the role names on the two ends are usually different, often two 'ends' of a related concept. Figure 8 shows a fragment containing role names for *Guest* and *Room* classes in a model of a hotel.

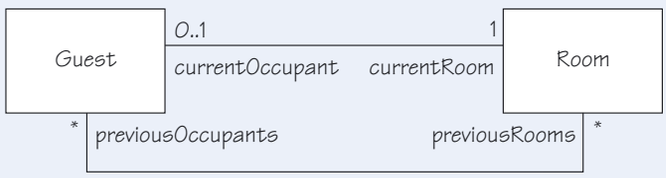

Figure 8 Role names

The placement of role names can be confusing. A role name for an associated object is placed at the opposite end of the association in which the object plays the role. For example, in Figure 8, one of the roles that a guest plays for a room has been named *currentOccupant*. From the point of view of the room, the guest on the end of the association is the *currentOccupant*. As you can see in the figure, the role name is at the *Guest* end of the association.

Assumptions about meaning

As with all parts of the notation, it is important to give a symbol precisely the meaning that is defined, rather than to make assumptions and read some other interpretation into it. It is extremely easy to give meanings to symbols that were not intended. For instance, consider the model in Figure 9, which says that each *Person* has a single associated *birthCity* and a single associated *workCity*.

Figure 9 Two associations

If the model had not included a *workCity* but had mentioned only the *birthCity*, there is a strong possibility that you would have used your knowledge of English and your understanding of life to assume that the model would not allow the birth city to change during the lifetime of a person. There is no such temptation with the association of the *workCity* since people take jobs in different cities at different times. However, there is nothing in the model that permits you to make either assumption. Both of these associations have been represented with the same notation. It is because of the meaning that we as humans attach to the role names that you are tempted to interpret one as a fixed association and the other as variable. If you want the model to capture the unchangeable aspect of an association, something more than multiplicity is required. You will see how to express such constraints later.

Navigation expressions

Once you have assigned role names, it is easy to refer to attributes of related objects, because roles can be used in a fashion similar to attributes. Figure 10 shows a simple class model and an object diagram corresponding to it.

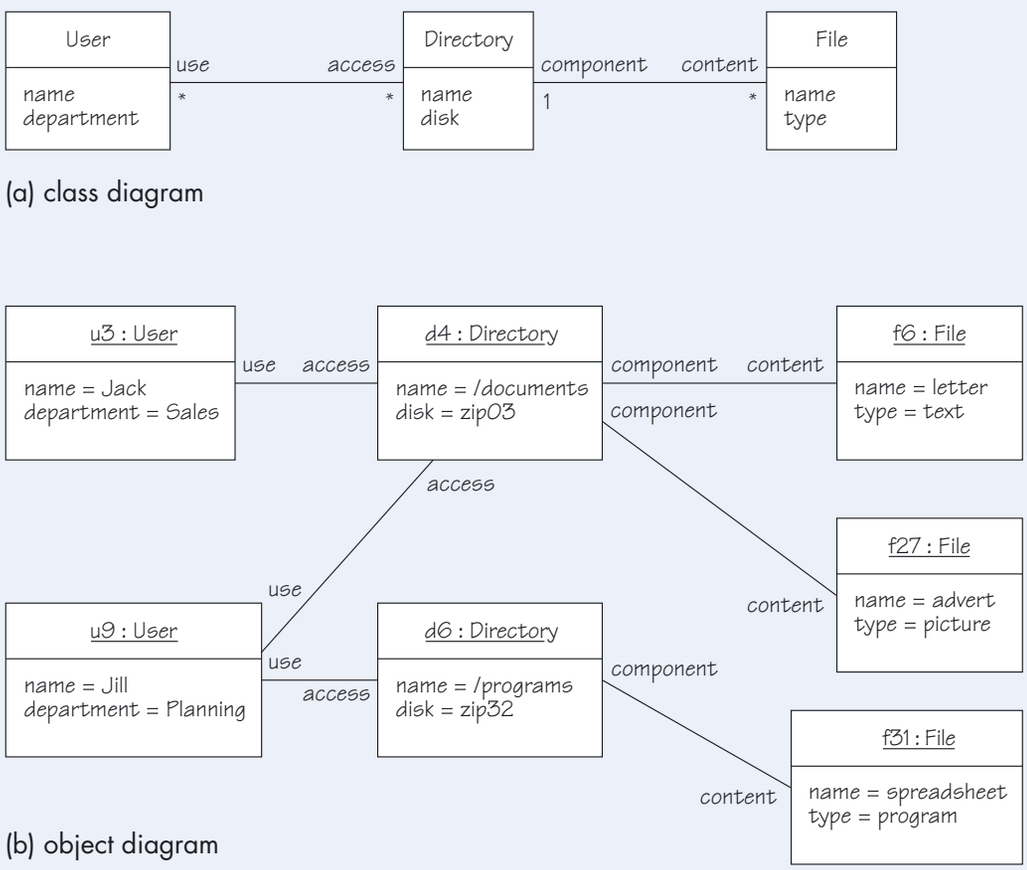

Figure 10 Role names in object diagrams

Navigation expressions are part of OCL, which is introduced more fully in Section 5.

In this figure, the identifier *f27* refers to an object of the class *File*, so f27.name will refer to the *name* attribute of that specific object. Using role names to navigate between objects, f27.component refers to the *Directory* object *d4*, and f27.component.disk refers to the *disk* attribute of the *d4* object of the class *Directory*, using the *component* role name. The term **navigation expression** is used for this kind of notation, in which you name an object or object attribute by starting at some object and then hopping from that object to another. Navigation expressions are naming conventions; it may be that the actual objects do not know about each other or contain attributes permitting the navigation. When using a navigation expression, you start at one object, and then use role names to identify the next object or attribute of interest. If there is no role name at the other end of an association, you can use the name of the class at that end of the association, but with the first letter in lower case.

The first part of a navigation expression identifies the object where the hopping is to begin. The middle parts, if any, are role names that you use to indicate which objects connected by associations are next. The expression may end with the name of an attribute, in which case you are referring to that attribute, or it may end with a role name, in which case you are referring to the object (or set of objects) connected by the association being used. The expression may consist of a single object name, in which case you are referring to that object.

Here are some examples from the hotel models (see Figures 2 and 8):

- jack — the starting object of the class *Guest*;
- jack.currentRoom — a *Room* object reached by following the *currentRoom* role name;
- jack.currentRoom.floor — an attribute of the *Room* object.

Several associations between a pair of classes

There can be more than one association between any two classes, as illustrated in Figures 9 and 11. Figure 11 can be interpreted as saying that each shop knows about one set of customers called *allCustomers*, another set of customers called *thisWeeksCustomers*, yet another set called *valuedCustomers*, and a single customer called *bestEverCustomer*.

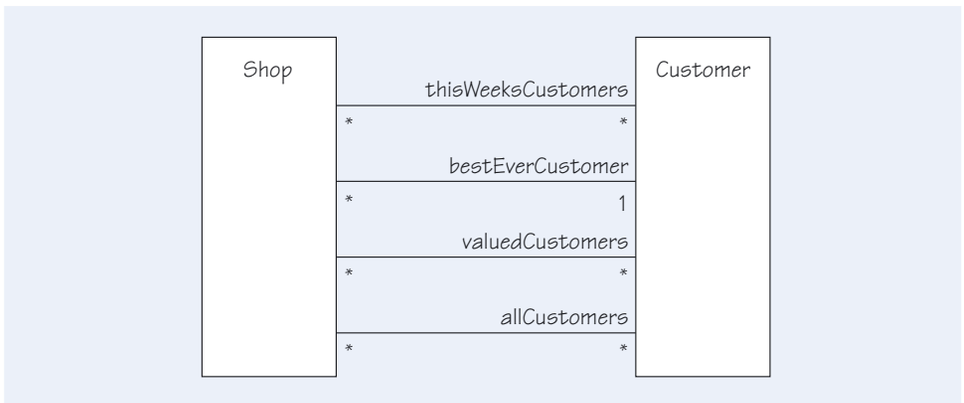

Figure 11 Several associations

As a modeller, you will want to represent the structure of the domain; if there are several associations between two classes in your model, you will need to include them all. Do not allow implementation considerations to sneak in at this point. You might be tempted to describe the *bestEverCustomer* as an attribute because you believe that is how it will be implemented, but from a modelling perspective, that would be incorrect. You are modelling the relationships that a customer might have with the shop, not how they are to be implemented. It is vital that you do not let your programming knowledge of *how* to represent things corrupt your representation of *what* it is that you are representing.

Associations between a class and itself

So far, all associations have been between different classes, but you often need to represent an association between a class and itself. A *Person* can be friends with other people (more than one *Person*) or work for another *Person*. Such associations are called **recursive**. Figure 12 shows a recursive association.

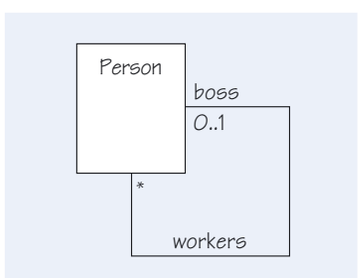

Figure 12 A recursive association

The figure shows that a person can have at most one boss, and that a boss may be responsible for several workers. Note that the role names are necessary to distinguish the meanings of the ends of the association. As the ends have different multiplicities, you must distinguish them; otherwise the model could be interpreted as allowing multiple bosses and at most one worker. Note that although common sense might tell you that a person cannot be his or her own boss, there is nothing in the model to rule

this out. If this is not an allowable state of affairs, you will need to express this constraint in some way other than by using multiplicities on associations. You will see how to do this later in this unit.

Recursive associations may or may not be symmetrical. If *jack* is a colleague of *jill*, *jill* is also a colleague of *jack*. However, *jack* can *like jill*, without *jill* liking *jack*. Being a colleague is symmetrical. Liking a colleague is not necessarily so.

SAQ 5

(a) Does a multiplicity of `1` indicate that there can be no change in the object to which the multiplicity relates?

(b) If an airline system models flights and pilots, and each flight needs two pilots, would you use a multiplicity of `2`?

(c) Suppose that each person has a number of wardrobes, and each wardrobe contains an even number of shoes. How would you model the evenness of the shoes?

(d) If a model contains role names, do you also need to use association names?

(e) What is a navigation expression used for?

(f) What is a recursive association?

ANSWER.....

(a) No. It merely means that at any one time there will be exactly one object at that end of the relationship. The attributes, or even the identity, of this object may change over time.

(b) Probably not. There are probably times during the life of a *Flight* object when fewer than two pilots are allocated, such as when the flight has been scheduled but crew details have not yet been settled.

(c) You might use a multiplicity on the association between the classes *Wardrobe* and *Shoe*, indicating that valid values were `0`, `2`, `4`, `6`, `8` and so on up to some reasonable limit. Alternatively, you could say that a *Wardrobe* contains an arbitrary number of instances of a class called *ShoePair*, where each *ShoePair* contains one left shoe and one right shoe. This approach generalises more easily to situations where the groups are not homogeneous. For example, a table setting contains one knife, one fork and one spoon.

(d) No, but it is sometimes convenient to have a name for the association as a whole. For example, you might focus on what is meant by *works for*, rather than the need to consider both the role *employer* and the role *employee* (at the same time).

(e) It provides a way of naming another object or its attributes relative to a starting object, by referring to intermediate role names.

(f) A recursive association is an association where both ends terminate at the same class.

2.11 The hotel chain revisited

You have already seen how noun identification can be used as a means of identifying concepts that will be used in your early class diagrams. Earlier in this section, we suggested that a wider view of the problem description is needed to get at the key concepts that will help you construct your first class diagram *en route* to specifying a software solution. Some concepts will become the key domain abstractions – the classes; others will define the relationships between the classes – the associations.

Figure 13 shows the key concepts that have been identified so far in the hotel system. However, it is not clear how we might deal with the fact that guests must have a reservation before they can check in.

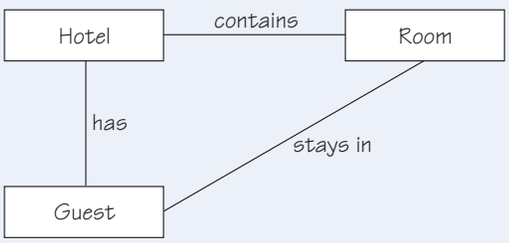

Figure 13 A set of key hotel concepts

As a separate activity, you may want to compare Figures 13 and 14 for coupling and cohesion.

One possibility is to make the concept of reservations into a class and make it the focus of a more elaborate class diagram. Figure 14 shows how we could refine our first attempt and include the new information. The new class, *Reservation*, makes the connection between a guest and a possible room at a particular hotel. We use a class because we want to model a relationship involving three things: *Guest*, *Room* and *Hotel*. This cannot be done with individual associations between *Guest*, *Room* and *Hotel*, because associations relate two classes. If we had three associations, they would be independent of each other, which is not how reservations behave. Note that the change has resulted in three roles for the class *Room*: *allRooms*, *allocatedRoom* and *currentRoom*.

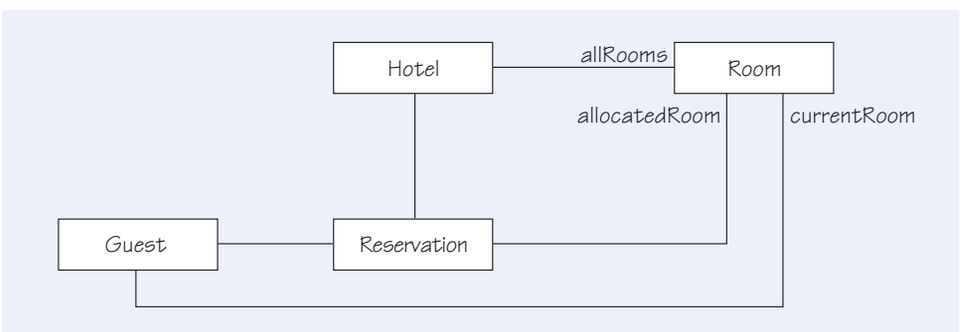

Figure 14 A first refinement of the hotel concepts

Figure 14 does not capture the fact that we expect each guest to pay for his or her stay. One way to do this is to create two new classes *Bill* and *Payment*, and create new associations between them and *Guest*. However, a large hotel chain is likely to offer services other than simply providing rooms for guests to sleep in. For example, each hotel would like its guests to use the restaurant facilities. Consequently, each guest's bill may have the charges for several meals on it as well as the cost of any particular room. Following the principle of modularisation, we would expect to deal with bills and their payment in a separate area of the software system – a separate subsystem package, to use the correct term from UML.

We still need some way of informing potential guests of the cost of any given stay and the invoicing methods for the different rooms at a particular hotel. One way to do this is to create a new class, *RoomType*, to deal with the common characteristics or properties of different rooms. We will assume for the purpose of discussion that *RoomType* has an attribute called *dailyRate* to help us generate an offer and open a new bill when a guest checks in.

Figure 15 shows some further refinements of the concepts shown in Figures 13 and 14. It includes the multiplicities for each association as well as some potential attributes. Notice how the class model has changed with respect to the concept of reservations.

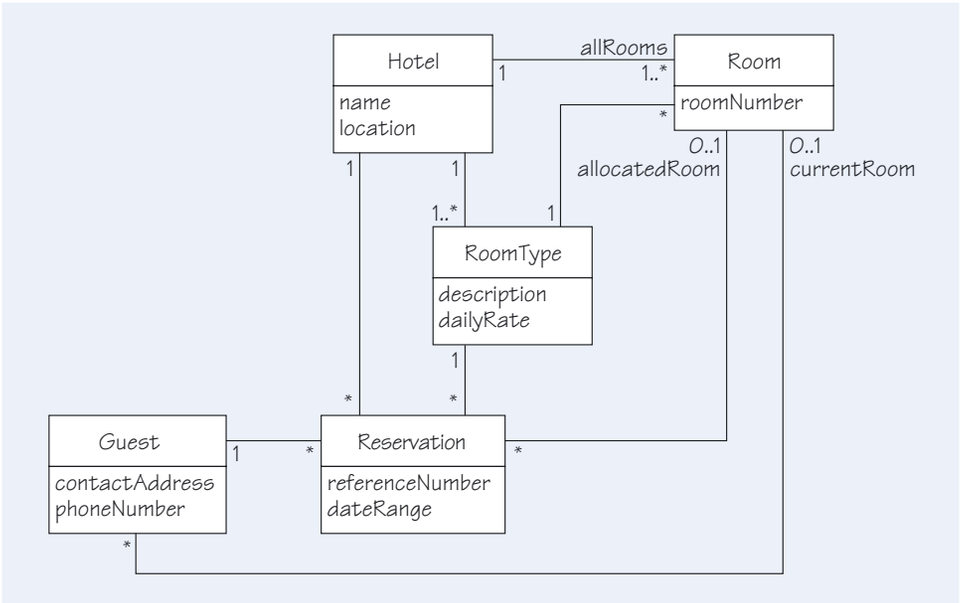

Figure 15 The classes for a hotel chain that requires reservations

As we learn more about the problem domain and the requirements, we expect to review and revise our models. For example, we may decide to change the class diagram shown in Figure 15 if there is no need to differentiate between the planned and the actual occupancy of a room. In the next section, you will see how such additional information can be shown on a class diagram.

Exercise 1

Imagine you are drawing up the specification for a software system for a petrol station to administer the dispensing of petrol and its associated billing. Consider all the interactions that are involved between driving into the petrol station and departing. What makes the petrol flow? How does the cashier know how much to charge? How does the manager know when to refill the storage tanks?

Make a list of all the nouns that you use in describing how various people make use of the system. (You are not required to make judgements about relevance or implementation at this stage.) After you have done that, filter your list of candidate classes. You need to be clear about the reasons for rejecting them.

Solution

Here is our list, but yours may be different.

arrival	bill	car
car's petrol tank	cashier	change
cheque	credit card	customer
delivered volume	departure	fuel cost
fuel delivery	fuel quantity sold	fuel type
holster	manager	nozzle
petrol tank cap	price display	pump
sale	signature	storage tank for fuel
trigger	unit fuel cost	volume display

You can group the rejected nouns from our candidate class list using the following reasons:

- out of scope – customer, cashier, petrol tank cap, cheque and signature;
- probably out of scope – change and credit card;
- an operation – sale, arrival, departure and fuel delivery;
- attributes of something else – car's petrol tank, unit fuel cost and fuel type.

Exercise 2

Table 1 records some of the associations and their multiplicities contained in the class diagram in Figure 6. Complete the table with the remaining associations, stating what their multiplicities might mean (look at the role names for clues). Are there any multiplicities that you think are unrealistic and that you should review with the help of a domain expert?

Table 1 Some associations from the hotel model

Association	Class A	Multiplicity for class A	Class B	Multiplicity for class B	Comment
location	*Hotel*	`1`	*Room*	`1..99`	Every hotel in the chain has at least 1 but no more than 99 rooms. Each room is in precisely 1 hotel.
occupancy	*Guest* in the role of *currentOccupant*	`0..1`	*Room* in the role of *currentRoom*	`0..1`	Each room is either empty or has a guest in it. Each guest is either currently occupying a room or not.

Solution

Our solution is shown in Table 2.

Table 2 Further associations from the hotel model

Association	Class A	Multiplicity for class A	Class B	Multiplicity for class B	Comment
reservations	*Hotel* in the role of *hotelReservation*	`0..1`	*Guest* in the role of *potentialGuests*	*	Each hotel has *many* guests (not all of whom have actually stayed in the hotel). Each guest may have made a reservation with a single hotel or may not yet have made a reservation.
history of occupancy	*Guest* in the role of *previousOccupants*	*	*Room* in the role of *previousRooms*	*	Each guest has previously occupied *many* rooms (some guests may not have occupied a room at all – yet). Each room has had *many* guests occupying it (although some rooms may not yet have had anyone staying in them).

The idea that each room can be occupied by at most one person seems rather restrictive. Generally, a hotel would have different types of room, such as single, double and twin rooms, and suites, some of which could hold more than one guest. Some hotels that we are aware of do restrict the number of guests per room, to a maximum of 4, so this is a good area for further investigation.

We think that a *Guest* should be allowed to have reservations for more than one hotel (think of booking a multi-centre holiday, or reserving rooms for a sales representative who travels a great deal). Similarly, a *Guest* should be able to make several reservations for the same hotel at different times. Beware of the temptation of including a restriction that a *Guest* should not have multiple reservations that overlap in time, since this assumes that the person who makes the reservation is the person who finally occupies the room. The notion of *Guest* being a person who has contacted one of the hotels in the chain to make a reservation needs further investigation.

On reflection, we think that an individual could have several roles in relation to a hotel, for example, as a customer who makes reservations and pays for the service, as a lodger who occupies a room, and as an enquirer who has asked about the hotel chain's services but has not yet entered into an agreement to receive a service. Therefore a single class *Guest* is probably not sufficient in a model of a real hotel system.

Exercise 3

In Figure 1, suppose that Jack checks out of the hotel. What changes would you make to the diagram?

Solution .....

The link between *jack* and room *r123* should be deleted. Whether or not the link between *jack* and *theRitz* should be removed needs to be checked with the domain expert, because the hotel may wish to retain some link with its guests after they have checked out.

Exercise 4

Give examples of different possible interpretations of classes with the names *Room* and *Guest*. When would new instances be created, and when would they be destroyed?

Solution .....

A *Room* class might represent physical rooms, which might be created and destroyed as building operations change the number of physical rooms. The instances might represent lettable rooms, whose existence might be related to whether the room was unlettable because of cleaning or repair works. A *Guest* class could represent a person currently staying in the hotel, which would be created at check in and destroyed at check out. It might represent a person who has stayed at least once, so that they would be created on the first check in and not destroyed on check out, but kept on the books. It might represent a potential guest, such as someone who has reserved but perhaps did not stay. Instances would be created on first contact with the hotel, and perhaps would not be destroyed, or not until they had generated no business for some years.

Exercise 5

Figure 16 presents a preliminary model for a software system to assist with the administration of a number of orchestras. Study the model and make a list of possible problem areas that would need to be resolved with the domain experts – presumably the orchestra managers.

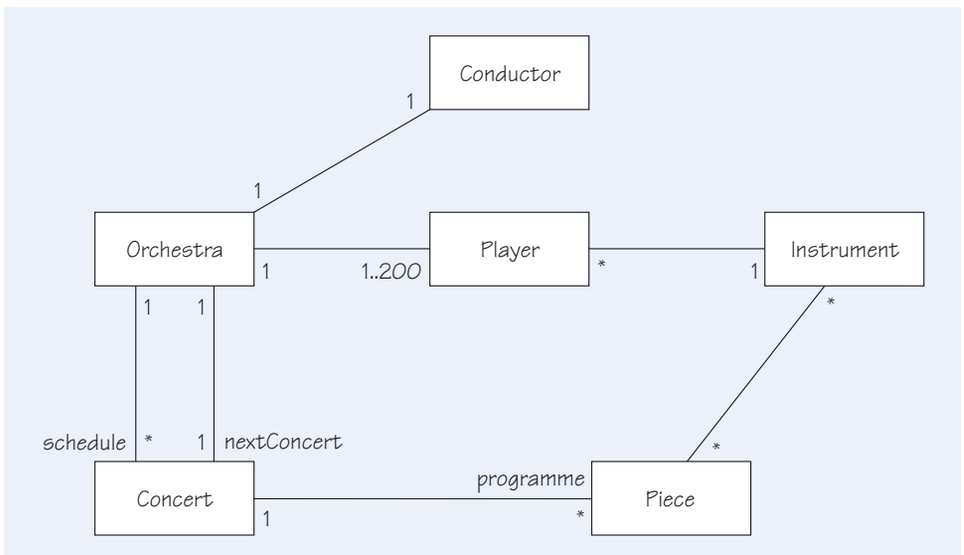

Figure 16 A class model for an orchestra

Solution

You may have identified other problem areas, but here are nine that we found using some domain knowledge.

1 A player can play for only one orchestra.
2 The model cannot represent an orchestra that currently has no players, such as during the gap between forming the orchestra and hiring the first player.
3 Is there always a 'next concert' – even immediately after a new orchestra has been created?
4 Does any player ever play multiple instruments?
5 What does 'instrument' mean? Does it refer to an instrument type, such as a violin, or to a particular instrument, such as Susan's Stradivarius violin? If it means the latter, do we need the concept of 'instrument type'?
6 The model currently just shows that pieces of music need instruments. Do you need to say how many of each instrument will be needed?
7 Does a single orchestra give each concert?
8 Is there only one conductor associated with an orchestra? Do you need to distinguish chief conductors from guest conductors? Should *Concert* have an association with conductors?
9 What is meant by 'piece'? How many times can an orchestra play each piece?

Exercise 6

Build a model to show the relationships between a person and their natural parents. Does your model prohibit someone from being his or her own mother? Is your model true for the whole of humanity or just for the people represented in some software system?

Solution

Figure 17 shows one possible model. There is nothing to say that the people at the end of the relationships are different. Since the non-identity of parent and child is part of the meaning of being a parent, you will need to capture that constraint in some other way. We have shown that every person has a father and a mother. That is true if we include dead people as instances of *Person*, but is not true if *Person* represents a living person, or a person in some finite set such as those represented in a computer. This is another example of how vital a project glossary is to relate terms in the class model, such as *Person*, to one particular meaning in the world. Simply naming a class *Person* is never enough.

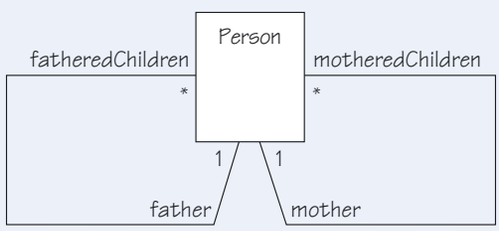

Figure 17 A model of families

2.12 Summary of section

This section introduced the basic ideas of class modelling. You have seen that techniques such as noun identification can produce a list of candidate classes from a description of the requirements. Such a list should be filtered to remove any inappropriate classes, such as those that are beyond the scope of the proposed software system.

Once a suitable list of classes is identified and recorded in a glossary, UML can be used to represent the classes, their attributes and their associations in a model. You have seen how to express the multiplicity of the classes involved in a given association, and that multiplicities cannot express all the constraints concerned with an association. Furthermore, you have seen the benefit of using role names over the simple identification of associations using verbs.

Considerations of the lifetimes of objects and their links often reveal inadequacies in the class model. In particular, the fact that the class of an object is fixed for its lifetime when it is created, and can never subsequently change, places considerable constraints on how you use classes to model a domain.

3 More on associations

This section introduces some additional concepts that will allow you to increase the precision and rigour of a model by focusing on the associations between classes.

By now you have seen enough features of UML to do some useful modelling of the classes for a potential software system, but there are many aspects of a model that cannot yet be expressed. If they are not expressed in some way, either in UML or in English, different readers of the diagram will form different assumptions. If people make different assumptions, there is a danger that the wrong software will be built, or that the software will not work at all. You will need more precise modelling concepts, which are the subject of the remainder of this unit.

UML has a rich vocabulary of notation that you can use in your modelling activities. Different groups of modellers will use the notation in different ways. Some prefer to use as much of the language as possible, while others prefer to use a small subset. Our aim is to introduce most of the diagrams in UML, and explain their role in software development; it is up to you or your company to decide which diagrams to use on a given project.

3.1 Interpreting attributes

At different stages in modelling, you will use classes to represent different kinds of information. During conceptual modelling, you might be building a model to explain how a complete hotel works, with no particular computing system in mind. The classes would represent pure concepts, rather than bits of software. They represent the more tangible business objects and their properties. The attributes of such classes represent parts of the domain expert's vocabulary. For instance, a hotel manager will use terms like daily rate, occupied and en suite when discussing rooms. These terms must be modelled somehow.

When you decide to build some software to automate part of the hotel's operations, the specification model you build will again include classes, but now the attributes will represent something that is defined in a software object.

SAQ 6

When considering attributes, what is the effect of moving from a conceptual model to a specification model?

ANSWER.....

The conceptual model records attributes of classes that will be familiar to a domain expert. For example, a hotel manager will be familiar with the daily rate for a room and whether or not it is occupied. In the specification model, the developer must consider the representation of attributes within a software system. For instance, daily rates for rooms involve money, and you can use a true/false (Boolean) expression to represent the occupancy of rooms.

3.2 The importance of time

A class model is meant to express all valid configurations of the system. It is possible to build a model that appears to be true most of the time, only to find that you have overlooked and accidentally excluded certain valid states. Figure 18 shows a fragment of a model that expresses the fact that an invoice has at least one entry (known as an invoice line).

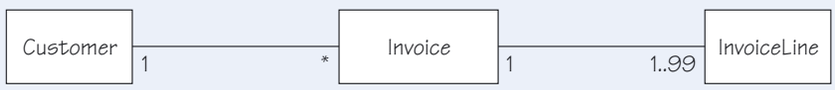

Figure 18 Invoices

This might look sensible, as a company would not normally send out an invoice that had no lines. However, if you consider the entire life history of an invoice, you may discover that there are circumstances in which it contains no invoice lines. An invoice may be created when an order is placed, but the invoice lines are only populated as the items are picked or manufactured. It could be that a zero-value invoice is created when an account is closed. Similarly, during editing, all lines might be deleted before new lines are added, and the invoice may remain in this state for some days. It seems that the model is probably wrong.

Very often, consideration of the life history of a relationship reveals other associations that should have been modelled. Figure 19 represents part of a library system that maintains an association between members and books.

Figure 19 A single borrowing association

If you consider the association with the role name *loans* and ask about its life history, some issues arise. When are instances of such an association (links) created, and when are they broken? A link is probably made when the member borrows the book, but should it be broken when the book is returned? If the purpose of the association is to know which books are currently on loan, the association will need to be broken when the member no longer possesses the book. However, if you want to record which members have borrowed which books in the past, the association should never be broken. Such a quandary usually reveals that you have omitted an association from the model. In this case, you probably need two associations, with roles such as *currentLoans* and *pastLoans*. When a book is returned, it might be removed from one association and added to another. Figure 20 shows a slightly richer model, which has captured this distinction (although it does not show when the associations are created and destroyed: you need dynamic models to do this).

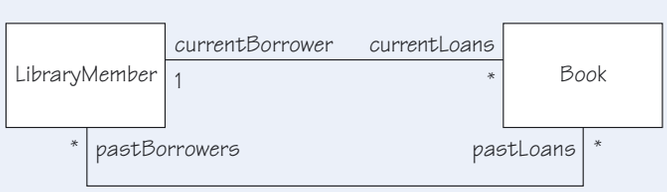

Figure 20 Multiple borrowing associations

A useful check on any model is to examine every element – every class and every association – and ensure that there is agreement with domain experts on *when* instances should be created and destroyed.

SAQ 7

Why is a class model not sufficient to describe a system?

ANSWER.....

A class model is a static model that describes the elements of a system (the classes) and their relationships (the associations), but does not describe the behaviour of the system over time. For this you will need one or more dynamic models. In particular, you need to model the life histories of objects and the interactions between them. The model needs to capture when instances of classes should be created and destroyed.

3.3 Aggregation and composition

An association represents the fact that an instance of one class is linked with a set of instances of another class. Figure 18 showed that an *Invoice* can have several instances of *InvoiceLine*. That is, there can be several invoice lines associated with a single invoice. We can describe this situation as a whole–part relationship: the whole (an invoice) is made up from the parts (the invoice lines). A similar **aggregation** association holds between an honours degree and the courses that were studied. In both of these examples it would be possible to add, remove or replace one of the 'parts' and still have a meaningful relationship. An invoice would still be an aggregation of invoice lines if another invoice line were added.

Aggregations have one very important additional constraint, compared with normal associations: aggregations must be free of cycles. If you arrive at an object by traversing through an aggregation association, there must be no way that you can continue to traverse associations away from the object and arrive back at that same object through any aggregation. More formally, aggregations must form a graph without cycles. Less formally, aggregations form a whole–part relationship, and no object may be part of itself, directly or indirectly.

There is a stronger form of aggregation called **composition**. In a composition, the composed objects are part of at most one composing object. If the composing object is deleted, all of the composed objects must be deleted. For example, consider a *Window* object (for a window on your screen) containing a *ScrollBar* and a *Button*. If the window is destroyed, the scroll bar and button must be destroyed as well. Compare this situation with a model for describing a *Product* consisting of instances of *Part*. One example might be the parts that make up a particular model of television set. A part, such as the screen, might appear in more than one model of television, or in none at all. Destroying one of the television model descriptions does not cause the description of the screen to be destroyed.

UML provides a notation to record the nature of a whole–part relationship. For aggregation, you simply add an open diamond to the end of the association to indicate the class that is to act as the whole. Composition, the stronger form of aggregation, is shown using a solid black diamond. Both notations are illustrated in Figure 21.

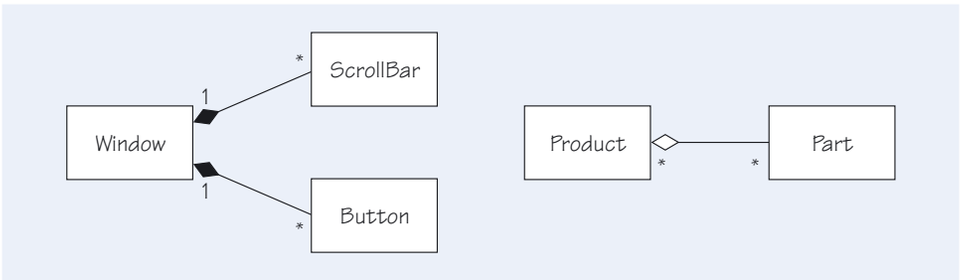

Figure 21 Composition and aggregation notation

3.4 Navigation

So far, we have not said anything about the direction of potential messages between instances of one class and instances of another where there is an association between the two of them. In Figure 19, for example, should a *LibraryMember* object be able to send messages to the *Book* objects that it is linked to? Should it be the other way around, or should we allow messages to pass in both directions?

In UML, you can put an arrowhead at one or both ends of an association to indicate that it is possible to reach one class from another following the direction of the arrow. In doing so, you are restricting access to instances of the class at the end of the association where you placed the arrowhead. As a convenience, if you put no arrowheads on the association, navigability in both directions is assumed. Figure 22 is a revised version of Figure 18. Now an *Invoice* can send a message to an *InvoiceLine*, but not the other way round.

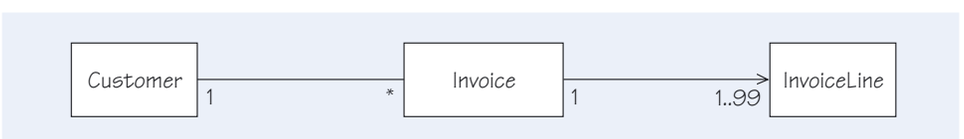

Figure 22 Constraining the navigation of an association

Note that navigability has no effect on navigation expressions. Navigation expressions are names used to indicate specific objects or attributes of objects, and do not take navigability into account. The notion of navigability is mainly of use during implementation, when you have to decide how to write the code that represents an association. If navigability can be restricted to one direction only (which is not always possible), the code will usually be simpler.

SAQ 8

(a) What is meant by navigability? When is this idea useful?

(b) In a multi-user operating system, users are allocated passwords. Draw a fragment of a class model to represent this association. Bear in mind that you do not want to be able to identify the corresponding user for a given password. What does this tell you about the representation of the association?

ANSWER.....

(a) Navigability means that it is possible to identify (or 'reach') objects in one class from objects in an associated class. The usefulness of this idea is realised during implementation when navigability in one direction alone (unidirectional navigability) can lead to simpler code.

(b) Users will want to 'know about' their passwords, not the other way round. Figure 23 shows that each instance of the class *User* will have a collection of references to the appropriate *Password* objects.

Figure 23 Navigability from *User* to *Password*

3.5 Qualified associations

A suitable time to generate a unique account number is when a new account is created.

Suppose that one of your classes has an attribute that acts as a unique identifier. For example, a bank usually identifies an individual bank account by a unique account number. Figure 24 shows how you might initially represent banks and their accounts, with an attribute to represent a bank account number. To ensure that the assumption that account numbers uniquely identify accounts is documented, this use of account numbers would be noted in the glossary.

Figure 24 Account with account number

UML allows us to model this situation by using what is called a **qualified association**. Instead of a *Bank* instance being linked to many *Account* instances, as in Figure 24, it is shown as linked to at most one *Account* per account number. Figure 25 shows such a qualified association. The *accountNo* attribute has been moved out of the *Account* class, and has become a qualifier.

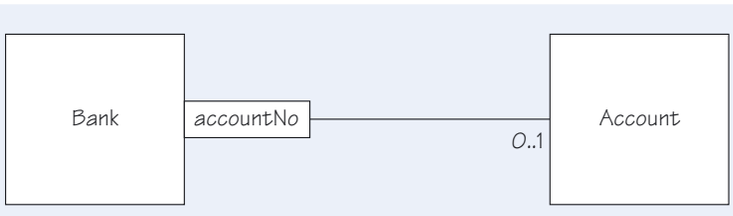

Figure 25 Using a qualified association

The reason why the multiplicity at the *Account* end is `0..1` and not `1` is that there may be account numbers that do not correspond to any actual account.

You can now read the model as saying that for a given combination of a bank and an account number, there is either one associated account or none. It tells you something extra about the *Bank–Account* association. The advantage of this approach is that you have put more information into the model, and have placed less reliance upon people using the glossary.

This illustrates the general point that a qualified association often replaces an association having a multiplicity of `*` with one having a multiplicity of either `1` or `0..1`.

In UML, a qualifier is considered to be an attribute of an association. It tells you about a property of the concept that relates the classes at the ends of the association.

In cases where objects can be uniquely identified by combinations of two or more attributes, all those attributes should move into the qualifier. For example, in a bank with many branches, an account may be uniquely identified by a combination of the account number and the branch number. The UK system of 'sort codes' and 'account numbers' works this way.

SAQ 9

(a) What is a qualified association?

(b) Suppose that, in the invoices example shown in Figure 18, invoices have unique numerical identifiers known as invoice numbers. How would you capture this information in a class diagram?

ANSWER.....

(a) A qualified association is an association at one end of which there is a qualifier, consisting of one or more attributes. The values of the attributes (taken together) uniquely identify the objects in the class at the other end of the association.

(b) You could use a qualified association whose attribute is named *invoiceNo* in a manner similar to that shown in Figure 25, by attaching the qualifier to the *Customer* end of the association between *Customer* and *Invoice*.

3.6 Attributes on associations

In some situations, it is convenient to put attributes directly on an association, instead of on one of the classes at the ends of the association. Consider a model for a chain of stores and the items on the shelves in each store. The owners of the stores want the price for an item to depend on the store it is in, and want to know the quantity of each item in each store. Adding an association between the classes *Store* and *Item* does not work, because there is no place to put the price and quantity. You cannot put it in the store, because that would imply a price and quantity for all items. You cannot put it in the item, because that would imply a price and quantity for all stores. The solution is to put the information on the association. Figure 26 shows this situation.

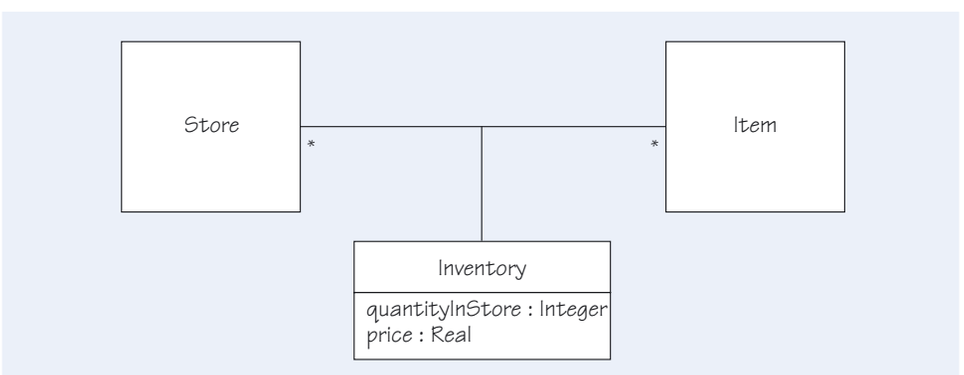

Figure 26 An association class

An **association class** imposes the restriction that there can be at most one instance of the class per pair of (non-association) objects. In the example shown in Figure 26, there can be at most one link from a given instance of *Store* (for example, the London store) to a given instance of *Item* (for example, a Tower of London souvenir cup). One

instance of the association class exists for each link between objects of the other two classes. This means that each link from an instance of *Store* to an instance of *Item* has the attributes *quantityInStore* and *price*, meeting the requirement.

3.7 Derived associations

As you work with your domain experts to develop your class models, you may find that you want to add an association to the model because it is important for understanding, but the association is redundant because other associations in the model can be combined to achieve the same navigation. To avoid this redundancy, UML provides **derived associations**.

Figure 27 shows some derived associations. Starting from a *Family* object, the association with the role name *allChilden* would produce a set of *Child* objects representing the children in the family. However, because the *Child* object contains the attribute *age*, you could easily compute a set of objects representing the teenagers in the family by first getting the set of all children, and then removing the children who are not between 13 and 19 years old. You could do something similar to compute the set of children living at home. So the *teenagers* and *childrenAtHome* associations are derivable from *allChildren* and information in the object.

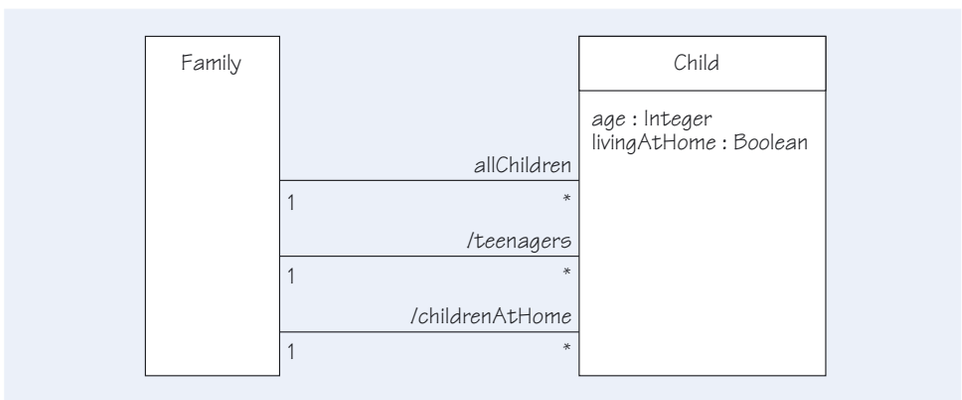

Figure 27 Some derived associations

By preceding a role name with a slash, you indicate that an association is derivable, although you do not say how it is derived. A description of how it is derived should be put in the project glossary.

It is not only associations that may be derived. UML allows you to derive a new element from the other elements in your model. Derived associations are probably the most common kind of derived element, closely followed by derived attributes. Any attribute can be marked as a derived attribute using the slash notation. For instance, a *Person* might have attributes of *dateOfBirth* and *ageInYears*. These attributes are not independent – if you were given *dateOfBirth*, you could calculate *ageInYears* – so the age could be marked as a derived attribute: */ageInYears*. Typically, the rule (the computation) by which one attribute is derived from another would be held in the glossary, but you could also express it in a note attached to the appropriate element in a model. For example, the value of *ageInYears* is obtained by comparing *dateOfBirth* with the current date.

There are several reasons for choosing to identify a derived element. As noted above, during analysis, a derived attribute or association may be used to identify and define a useful concept. The disadvantage is a redundancy in the overall model that requires an 'overhead' to be sure that the derivation rules are understood, possible, and valid after other changes to the model. Whatever the reason for choosing to identify a

derived element, its presence in a model implies that there is a responsibility to consider changing it if there is a change in the elements used to derive it.

Example 1

Subsection 2.11 describes a class diagram that represents the main concepts in the hotel domain for reservations and checking in and out. Figure 15 shows a direct association between *Hotel* and *RoomType*, and an indirect association through *Room*. As part of our design optimisation, we might find that the direct association is readily derived from the indirect association.

SAQ 10

Under what circumstances would you want to show an association that is not independent of others in the diagram?

ANSWER.....

If a word describing an association is part of the natural vocabulary of the domain expert, it will be sensible to include it in the model, as otherwise a linguistic gulf will start to open between the domain expert and the system designer. However, if you know that an association is not essential, because it can be derived from other associations, you will also need to record that fact.

Exercise 7

Build a class model associating companies, libraries and families with people, identifying people by suitable qualified associations. When considering a family, be sure that your model handles twins (multiple children with the same date of birth) and sharing of names between generations (for example, mother and daughter with the same name).

Solution.....

Looking from a company's and a library's perspective, you can find a way to identify a single person who may be a worker or a member, respectively, as shown in Figure 28. In the case of a family, two or more members of that family may have the same birthday or the same name, but the combination of name and date of birth should be unique. Using two qualifiers (*name* and *birthdate*) should uniquely identify a person.

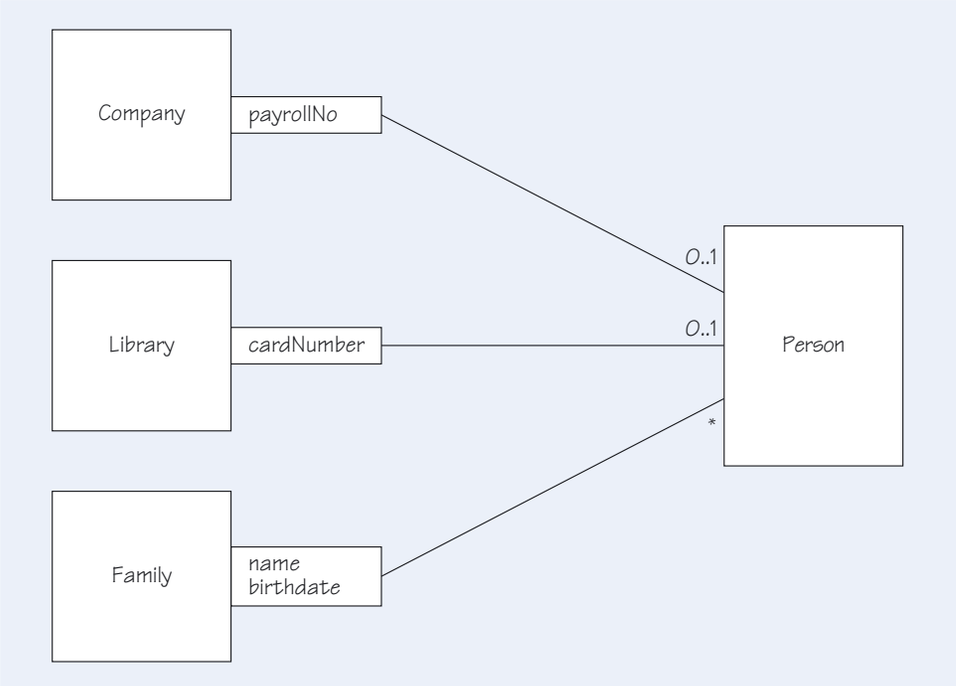

Figure 28 Model with qualified associations

3.8 Summary of section

You have seen in this section that class models are limited because they only show the overall structure of a proposed system. That is, a class model records the elements (classes) of a system and their relationships, but does not show the importance of time.

You have met other techniques that can greatly increase the precision and rigour of models. First, the navigability of an association can be used to indicate whether it is possible to traverse an association to obtain the object or set of objects of a given class. Second, qualified associations are a way of capturing the uniqueness of some manner of identifying objects, as in the case of account numbers in a bank. Third, it is possible to show additional, derived associations that are deducible from other associations in a class diagram.

4 More modelling techniques

This section will introduce you to additional modelling constructs that allow you to deal with specialisation and generalisation among classes. You will also see how the concept of an interface helps to control the dependencies between classes.

4.1 Generalisation and specialisation

Generalisation/specialisation is a relationship between two types. If type *B* specialises type *A*, an instance of *B* has all the features of an instance of *A* but in addition has features special to type *B*. In this case, *B* is a **specialisation** of *A*, and *A* is a **generalisation** of *B*.

Figure 29 shows a general notion of 'tree' and then two specific kinds of tree: oak and pine. There are some general properties and behaviour shared by all trees: they all have leaves and they tend to grow upwards (and outwards).

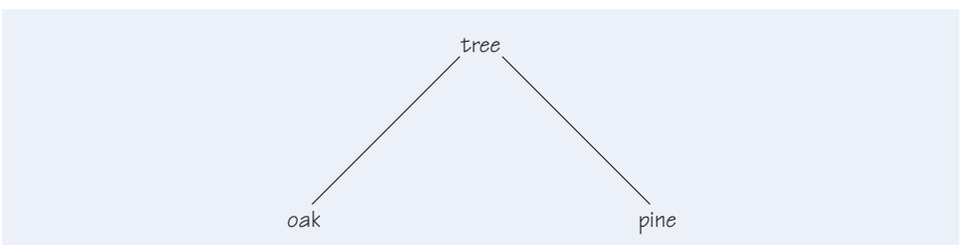

Figure 29 Different kinds of tree

Oak trees have identifiable differences in comparison with pine trees. For example, they have differently shaped leaves, and their behaviour is somewhat different. An oak tree drops its leaves in the autumn and grows new ones in the following spring, whereas a pine tree keeps its needles all year round. Note that we can only talk in more abstract terms about the tree at the top of the hierarchy in the figure. A tree is always a tree of some species. We do not see instances of a generic tree in our woods and forests, but we do see instances of oak and pine trees. We will return to the abstract nature of types at the top (or root) of a generalisation hierarchy later on.

Notice that statements may be made about a general property such as the direction of a tree's growth no matter whether there is an oak or pine tree in front of you. This illustrates, in a very simple way, that an instance of the more specific element can be used in place of the more general element. This is called **substitutability**: a more specialised element can be substituted for a less specialised element in the same hierarchy.

You may also be familiar with the terms *parent* and *child* for a superclass and a subclass respectively.

In object-oriented software development, we call the more general element a **superclass** and the more specific element a **subclass**. A subclass has the same attributes as the superclass, though it may define some additional ones. It also has the same operations as the superclass, though it may define additional operations. The natural time to decide on operations is once the requirements for the whole system have been agreed, and decisions are being taken on which parts of the functionality of the whole system belong with each class. Figure 30 shows a typical use of specialisation. We needed some operations and attributes so we can discuss specialisation – we wanted to be able to show subclasses having extra behaviour and

properties – so for this purpose we have supplied a plausible set of operations and attributes. How we would actually decide what operations are appropriate in practice is something we explore in a later unit. There is a class *Account* that represents the basic notion of something that maintains a balance, and supports operations to credit and debit the account.

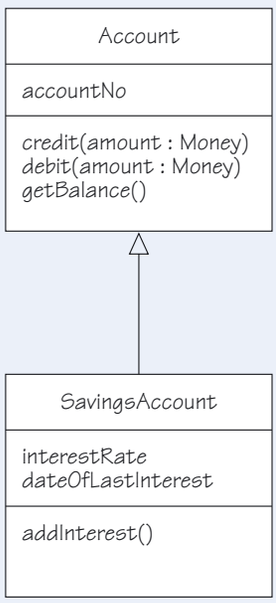

Figure 30 Specialisation of *Account*

Suppose that you already have an *Account* class and want to define a *SavingsAccount* class, with operations to credit, debit, get a balance and add interest. You could define a new class, from scratch, having all the necessary operations. If you did so, you would end up with two apparently unrelated classes. What would be missing is the reason that both classes have so many operations in common: a *SavingsAccount* really *is* an *Account*; it is just a special form of *Account* that can also handle interest. It is therefore better to model this connection directly. This is also better than defining the two classes entirely separately, because *SavingsAccount* will automatically share any changes that you make in *Account*, and you avoid duplication of effort.

We use the word interface for the set of operations that specify the service that a class provides. The class *Account* has the interface {*credit*(*amount : Money*), *debit*(*amount : Money*), *getBalance*()} while the class *SavingsAccount* has the interface {*credit*(*amount : Money*), *debit*(*amount : Money*), *getBalance*(), *addInterest*()}. Note that the interface of the subclass always contains the interface of the superclass, although in general the subclass will have extra operations as well. There is no mechanism for removing operations. You cannot define, say, a *BlindAccount* class as being a subclass of *Account* that does *not* have the *getBalance* operation. This fact is vital for substitutability, because it means that you can guarantee that all subclasses have the attributes and operations of the superclass, though they may well have more.

Subclasses can also redefine operations that are in the superclass. Figure 31 shows a different subclass of *Account*, called *GoldAccount*. This defines an extra operation, *addBonus*, which is not present in *Account*, but it also redefines the *credit* operation, which was present in the parent. Perhaps it redefines it to reward depositors by crediting an extra £10 to the account each time a deposit is made. The interface of GoldAccount therefore contains the operations *credit*, *debit*, *getBalance*, and *addBonus*. The class still supports all the operations of the superclass. It does so in a slightly different way, in that the behaviour of the *credit* operation has been changed.

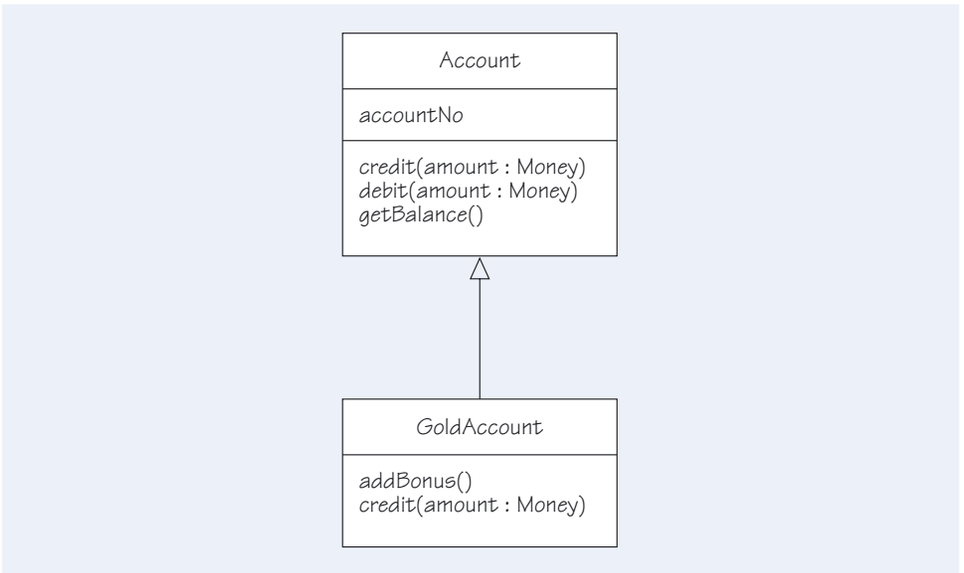

Figure 31 Another specialisation of *Account*

Specialisation can occur at various levels. Figure 32 represents a word-processed document containing various parts that are specialised at two levels.

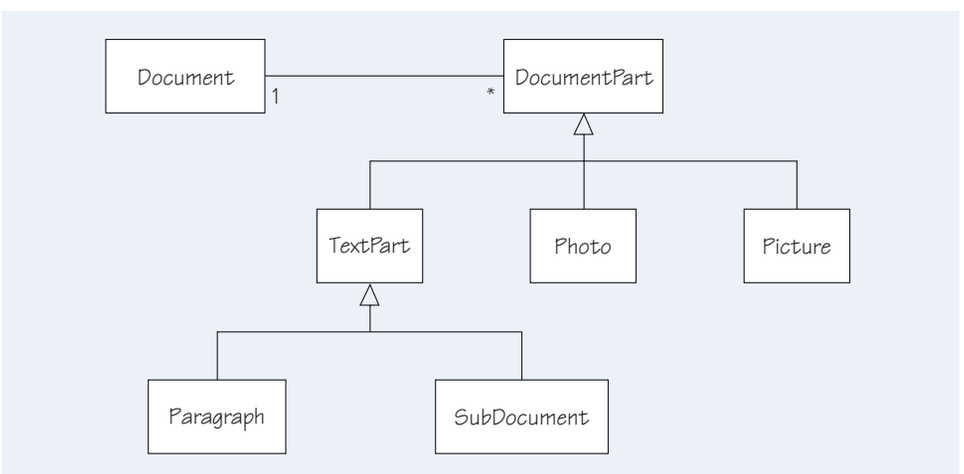

Figure 32 Multiple levels of specialisation

When deciding whether to model some distinctions by using specialisation, you need to remember that an object cannot change its class dynamically. Figure 33 looks like a reasonable model for representing the fact that children attend a school, and that adults are allocated to a tax office. You use two different subclasses of *Person*, and put the aspects that differ into those subclasses. However, this model will not be able to represent the change when a child becomes an adult. This may or may not be a problem. If the model is intended for short-term use, say in a school, it may never have to cope with such changes. Once again, you can see that whether a given model works or not can depend on the life cycles of the objects in the system.

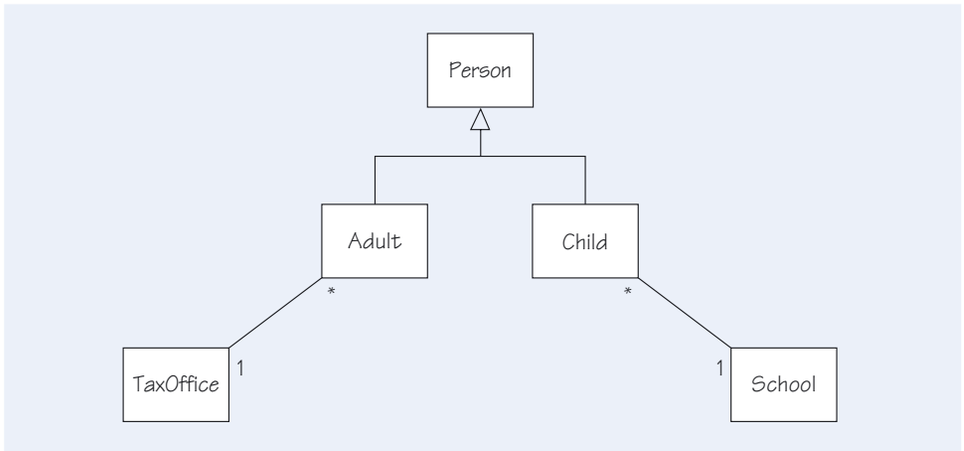

Figure 33 Specialisation used to distinguish roles

If you have several levels of specialisation, some associations in the model may be with the subclass, and others with the superclass. Figure 34 shows such a situation, where clubs know all about their members, but teams are related to playing members only. Both subclasses of *Member* inherit the association with *Club*, so an instance of either subclass can be linked to an instance of *Club*. However, only an instance of *PlayingMember* can be linked to an instance of *Team*. An *AssociateMember* instance cannot be so linked.

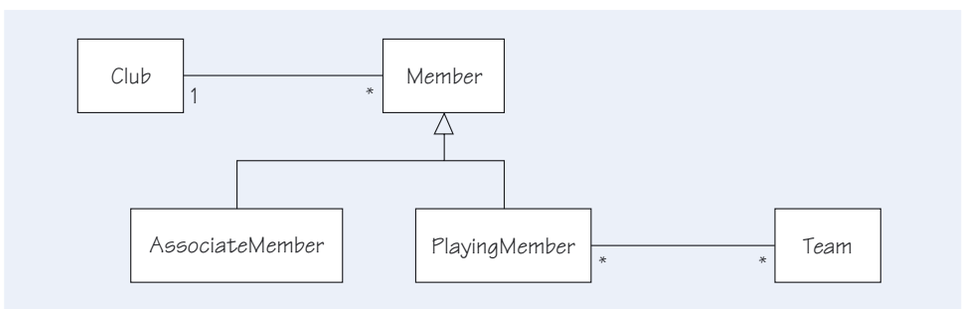

Figure 34 Clubs and members

The substitutability test can indicate whether a use of specialisation is correct. You should use specialisation only when an instance of a subclass is substitutable for an arbitrary instance of a superclass. Suppose a club is looking for a treasurer with the only constraint being that the treasurer must be a member. Any instance of *PlayingMember* would do, because a *PlayingMember* has at least all the properties of an ordinary *Member*; it also has some extra properties. Similarly, if the club wants an *Account* to keep its funds in, a *GoldAccount* will do, because this satisfies all the expectations one can have of a simple *Account*.

Note that *AssociateMember* is associated with *Club* but not *Team*. Instances of *AssociateMember* cannot be linked with instances of *Team*; they are not substitutable for instances of *PlayingMember*. If you want to play for the club, you have to be a *PlayingMember*.

Cases where the substitutability principle does not hold tend to arise when the use of specialisation is motivated by a desire to unclutter a model or by implementation concerns, rather than by analysis of the relationship between concepts.

Suppose you have an existing class *Car*, defined as in Figure 35. You now have a requirement to define a class *Bicycle*. You might notice that most of the desired attributes of *Bicycle* are also attributes of *Car*, so you decide to use specialisation to save some time. However some attributes of *Car*, such as *usesDiesel*, are inappropriate for a *Bicycle.* There is no mechanism for removing attributes in a subclass, so you leave them there and just hope that no one uses them. You have violated the substitutability principle because it is not true that a bicycle can be substituted for a car in all contexts. If you try to fill a bicycle with fuel, you will discover the differences.

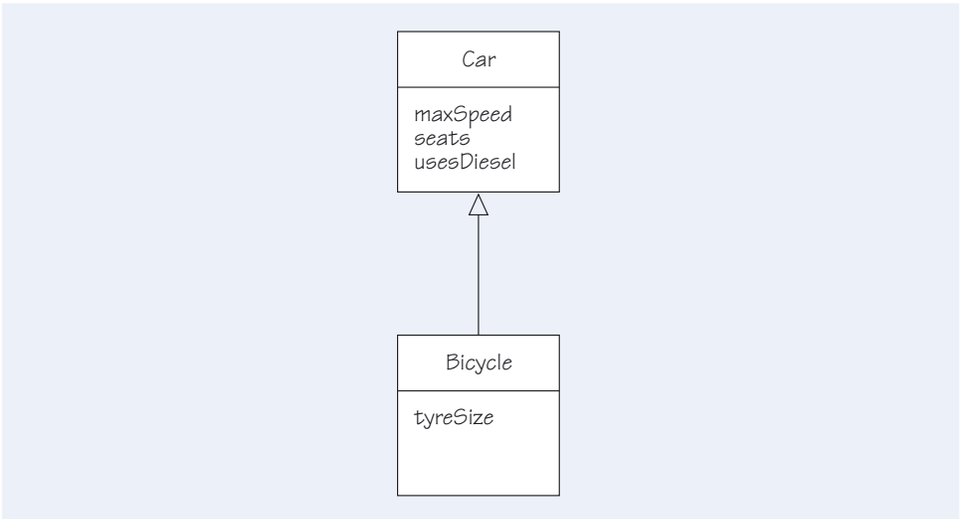

Figure 35 Improper specialisation

When tempted to use specialisation in this way, you should always investigate whether you should add a superclass to the model. A better solution is shown in Figure 36.

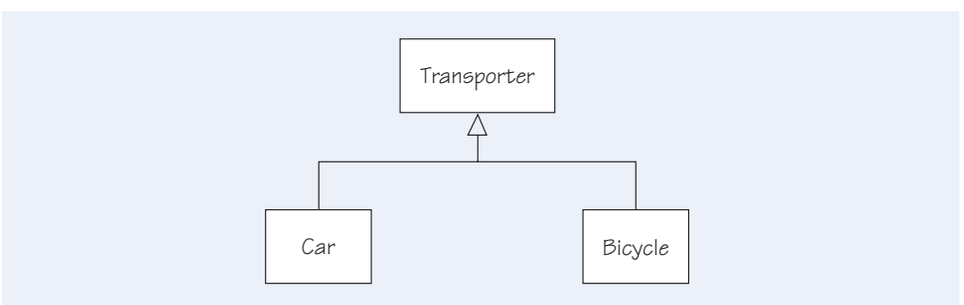

Figure 36 Inventing a superclass

You have invented a superclass, *Transporter*, but do not intend there to be any instances of it. There are no objects that are purely transporters – just cars and bicycles. The superclass exists just because it collects common concepts together and documents the similarity between the subclasses. The domain experts may not have thought of the superclass, but they should agree that it represents something, or at least does not break anything. Such a superclass will never be used to create instances, and is tagged in UML with the stereotype «*abstract*».

A stereotype is just a label that may be added to a model element such as a class or an association to convey extra information.

SAQ 11

(a) What is the difference between inheritance and generalisation?

(b) Look again at Figure 30. Will an instance of *Account* support the *addInterest* operation?

(c) In a computer graphical user interface, how might you represent the relationship between a full window and an iconised window from the classes *FullWindow* and *Icon* respectively? List the operations that are common to both and those that are peculiar to each.

ANSWER.....

(a) There is no difference between inheritance and generalisation. However, the term inheritance tends to be used by programmers, whereas the terms generalisation and specialisation are used by analysts and modellers.

(b) No; *addInterest* is not part of the interface for the class *Account*. It is, however, part of the interface for the class *SavingsAccount*, which is a specialisation of the class *Account*. Objects of the superclass cannot replace objects of any subclass.

(c) You could introduce a new, abstract class named *Window*, and let the other classes inherit from it as follows:

- abstract class *Window* with operation *display*;
- subclass *Icon* with operations *display* and *maximise*;
- subclass *FullWindow* with operations *display*, *iconise* and *scroll*.

Exercise 8

Figure 37 shows a simplified model of a lending library that holds copies of publications for loan by its members.

Figure 37 A lending library

(a) Using generalisation, draw a fragment of a class model to illustrate the fact that books and journals are both kinds of publication in a lending library.

(b) A typical lending library will stock copies of both books and journals. However, not all of these copies will be available for the members of the library to borrow. Some copies will be for reference only; they must not be taken out of the library. Copies for loan will have an attribute that indicates when they are due back in the library. After the due date, a fine is charged on a daily basis. Redraw Figure 37 to show the different kinds of copy that are held in stock, incorporating your answer to part (a).

Solution

(a) Books and journals can be represented as different specialisations of publication, as shown in Figure 38.

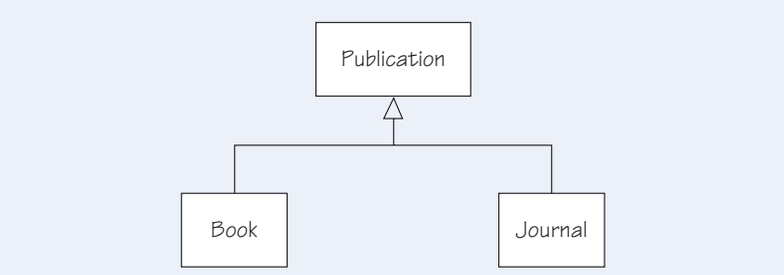

Figure 38 Class model for publications

(b) Following the same use of generalisation in part (a), we can use a hierarchy to show that copies for reference and copies for loan have different properties that we wish to model.

Our solution is shown in Figure 39, where we have added a *returnDate* attribute to the class *LoanCopy*, to indicate one of the differences between loan and reference copies.

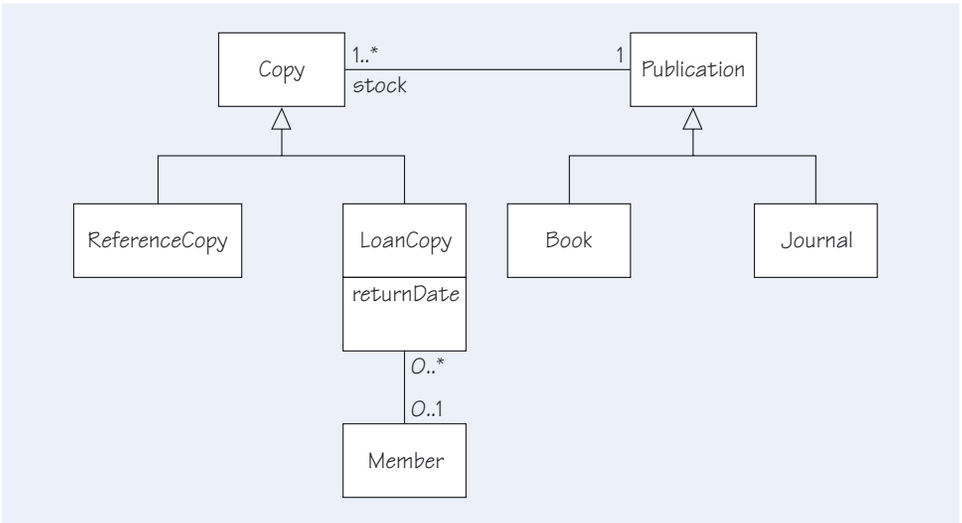

Figure 39 Class model for a library

As we have drawn the *Copy* hierarchy, it is possible to have instances of *Copy* that are neither for loan nor for reference. It is possible that people working in the library use copies of publications for their own purposes. If we wanted to exclude this possibility, we would make *Copy* an abstract class (showing this with the stereotype «*abstract*»).

The substitutability rule indicates a potential problem with the class model in Figure 39. For example, if we wanted to lend a reference copy to someone (for a special purpose such as a research project), this model would not allow it. If that is not what the domain experts want, then the model must be changed.

Exercise 9

The fragment of a class model in Figure 40 shows that there is a payment associated with a sale, which represents a common activity when purchasing goods.

Figure 40 Fragment of a class model

Suppose you are asked to model three different kinds of payment that are allowed at a supermarket checkout. The first method of payment allows a customer to use cash to pay for the items in their shopping basket. The second method of payment, which avoids the need to handle cash, involves a personal cheque that requires a cheque card to guarantee the amount to be paid to the supermarket. The third method of payment involves the use of a credit card, which requires authorisation by the credit card company.

(a) Draw a fragment of a class model to show the three different ways of paying an amount of money for a sale at the supermarket.

(b) Describe how you would model the details of the different payment methods. (Hint: think about subclasses.)

(c) There are three general reasons that we might have for adding subclasses to a model. See if you can work out what these three reasons are, by examining your answers to part (b) above.

Solution

(a) We have chosen to include a common attribute, called *amount*, in the superclass *Payment*. Figure 41 shows the three subclasses.

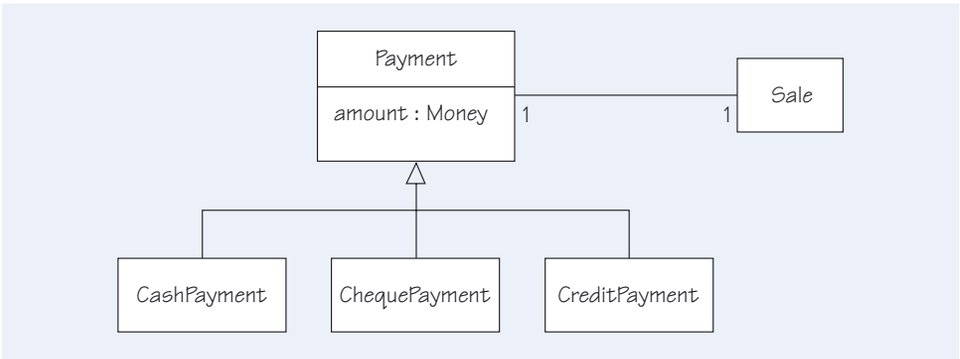

Figure 41 Class model for payments

(b) Although there are common features, each subclass of *Payment* needs to be dealt with in a different way. A cash payment is the simplest. A customer uses a combination of bank notes and coins to pay for the sale. Commonly, a customer hands over more cash than is required to cover the cost of a sale, so there is a notion of change to be handed back, which may have to be shown on the till receipt. The other two forms of payment are associated with the exact amount required, so no change is required (we will ignore facilities such as 'cash back').

A payment by cheque requires the customer to present a cheque guarantee card. You could choose to model a new association to a new class called *ChequeCard*, or you could simply use a new attribute to record whether a valid cheque card was presented.

Payments made by credit card are substantially different from the other two forms of payment because the supermarket must be confident that they will receive the required amount of money. In practice, this might involve a connection to the credit card company for authorisation.

(c) The above discussion illustrates that there are three situations that could lead to the decision to include additional subclasses in a model:

(i) there may be additional attributes of interest;

(ii) there may be additional associations of interest;

(iii) there may be different kinds of behaviour that ought to be represented.

That there are precisely three situations is related to the fact that classes have attributes, operations and associations.

4.2 Summary of section

This section has illustrated the power and dangers of generalisation and specialisation. You have seen that the creation of subclasses is a way of expressing the fact that elements share concepts. You have also seen that the substitutability principle should not be violated when considering generalisation and specialisation.

5 Constraining models

In this section we will introduce additional UML notation that makes it possible to capture aspects of the domain that cannot be expressed by the notation you have met so far. We look at how a model can represent facts about the domain which cannot be expressed just by classes, their operations and attributes, and associations with their multiplicities.

5.1 Constraints on classes

The UML notation we have described so far cannot capture all the static information about a domain. There can be many extra restrictions that should somehow be expressed. Of course, we could add all the necessary extra information to the project glossary, but in practice we would prefer to incorporate it in the diagram if possible.

First, we will investigate how to add these extra restrictions, or **constraints**, to the diagram. Then we will go on to look at how constraints can be identified. A constraint can be expressed in a natural language, such as English, or we may use a special notation defined by UML. It is also useful to introduce the concept of an invariant, something that must be true about the system at all times, because the constraints we will study are concerned with ensuring that the invariants are not disobeyed.

Some common kinds of constraint are:

- constraints on the values of attributes;
- constraints on associations;
- uniqueness constraints (which can sometimes be converted to qualified associations).

The simplest way to record constraints on a model is to attach a description of the constraint to the appropriate part of the model. In UML you can attach a constraint to a class by putting it within curly braces, as shown in Figure 42.

Figure 42 Constraints on classes

To say that a *Person* has an integer *age* attribute is to assert that any integer value is permissible, such as 123 or 999.

The attributes of the *Train* class, record the start and finish of a single journey of a train.

The contents of the braces can be in any appropriate notation. English is the most common, and for most purposes is perfectly adequate.

When considering constraints, it is best to think of them in terms of particular instances, just as you did when you used object diagrams to consider multiplicities. You could read the constraint on the *Train* class in Figure 42 as: 'for every journey,

each train has a departure earlier than its arrival'. However often the times may be modified, as trains are rescheduled, this model tells us something about how the modification must affect the times.

A constraint can be applied to any model element or list of elements. Each invariant is a constraint upon the possible configurations of objects in a software system. In general, a constraint defines a restriction that must be satisfied in any subsequent implementation. If your implementation breaks a constraint, it is incorrect with respect to the design.

In *Unit 6*, we introduce the concept of Design by Contract, which allows you to control and coordinate the constraints in your model and apply them to a subsequent implementation.

Constraints are also passed on from superclass to subclass, which introduces a dependency in a model. A subclass cannot ignore an inherited constraint, but it can modify the constraint by 'strengthening' it, that is, by adding further restrictions.

UML defines a formal notation called the **Object Constraint Language**, which combines logical expressions with set notation to allow a more rigorous specification of a constraint. As you will see below, certain constraints are predefined, while others may be defined by the modeller. In this course, we will usually express our constraints in natural language, but occasionally we will make use of OCL.

Finally, you should beware of the risk of introducing complexity into your class diagrams. If you show too much detail on a diagram, it becomes hard to read. It may be better to record only the most important constraints on a class diagram, and record the remainder in your glossary or other form of project documentation.

The question of how many constraints you need is certainly context-dependent. Just try not to get carried away and create constraints simply because you can!

SAQ 12

(a) Figure 24 shows the class *Account* with an attribute, called *accountNo*. Write an appropriate invariant that limits the values of this attribute to a number composed of a 3-digit branch code followed by a 7-digit identifier.

(b) When is an invariant on a class true?

(c) What are the risks involved when you try to record all the possible constraints on a model?

ANSWER.....

(a) We are not told anything about the range of either branch codes or identifiers, but we do know that a valid account number has 10 digits as follows:

{*accountNo* is a 10-digit number, with a 3-digit number at the beginning for the branch code, followed by a 7-digit identifier}

(b) The invariant on a class must be true for every object of that class from the time that object is created to the time it is deleted.

(c) The first risk relates to the complexity of the resultant model. If too many constraints are recorded on a model, that model will become difficult to read or comprehend. For each case, you should decide whether or not a given constraint adds value to your particular model. It may be more appropriately recorded in the glossary.

The second risk relates to the potential increase in the number and complexity of any dependencies that would arise for each additional constraint, especially those among two or more model elements. For example, if an association is constrained in some way, all other paths via association loops must be constrained in the same way.

5.2 Constraints across associations

Very often the constraint is not about a class, but is about the associations between classes. You have seen an example already in the discussion about qualified associations in Subsection 3.5. There are many different kinds of constraints on associations that you may want to express, and UML provides special-purpose notations for the more common ones. We will begin with the two standard constraints that describe the properties of pairs of associations.

Figures 43 and 44 show two ways of expressing the fact that the *companySecretary* must be a member of *staff*. Figure 43 uses the special notation {*subset*} (attached to a dashed arrow) to express this constraint.

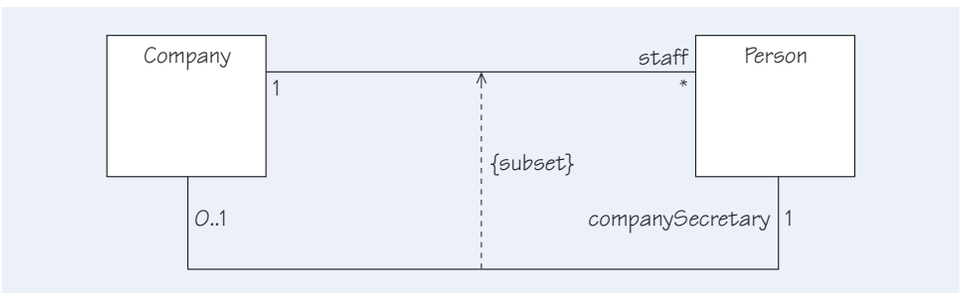

Figure 43 Formal constraints

However, if you take into account *who* is likely to read the class model, it may be more appropriate to add a note to the model explaining the constraint, rather than use a special notation. Figure 44 shows an alternative, informal approach in a class diagram that may be shown to a user. It uses a note to explain the constraint.

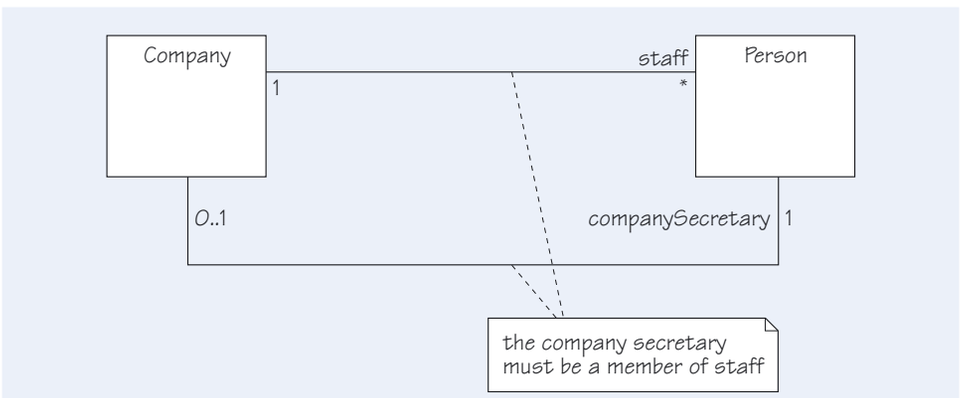

Figure 44 Informal constraints

Another standard kind of constraint applies to associations that are connected to the same class. The special {*xor*} notation is used to specify that an instance of the class must participate in exactly one of the associations grouped together by the {*xor*} constraint. To reiterate, one of the associations must always be active. The multiplicity of the active association must be respected, but none of the other multiplicities need be.

Figure 45 shows how this exclusive-or constraint can be used to express the fact that although the class *Customer* has two different associations, one with *SavingsAccount* and the other with *GoldAccount*, any particular instance of *Customer* will only have one of these associations.

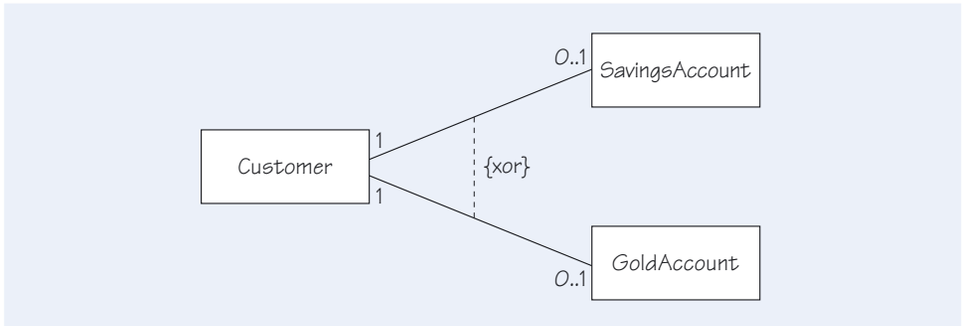

Figure 45 Controlling the use of different accounts

You have seen constraints on a single class, such as the examples in Figure 42, and constraints on associations, such as the examples in Figures 43, 44 and 45. Often constraints combine these elements. You might want to constrain the attributes of objects that are related by an association. In a family, we might want to capture the constraint that the ages of the parents must be greater than the ages of the children. In a company, there might be a constraint that an employee has a lower salary than that person's boss. You can express constraints such as these informally with notes, following the example in Figure 44. However, if there are a lot of associations on the diagram, it is useful to have a name for the objects related by each association. This is where role names can be particularly helpful.

If your modelling tool does not support constraints directly, place them inside a note. Then attach that note to the affected model elements.

Figure 46 shows one way to represent relationships between children and adults – as father, mother and teacher. Children and adults both have an *age* attribute.

In a different context, you might choose to model such relationships using inheritance.

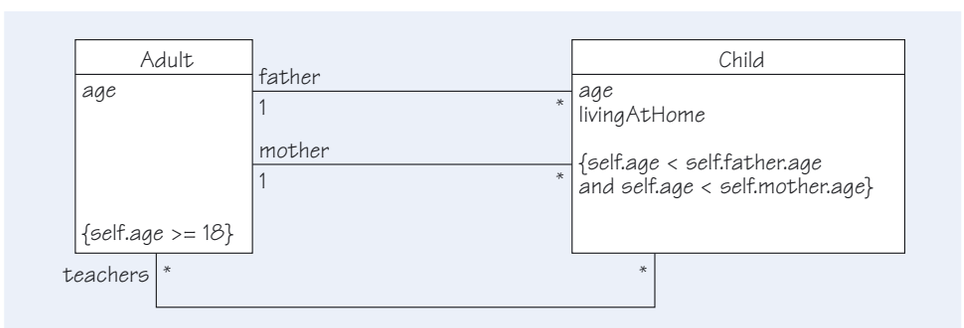

Figure 46 A constraint on related objects

To capture the common invariant that a child must be younger than both father and mother, we have written the following invariant for the class *Child*:

context Child **inv**:
self.age < self.father.age and self.age < self.mother.age

In Java, `this` is used instead of self.

Notice how navigation expressions, introduced earlier in this unit, are used to distinguish the terms or elements in a model. Given a typical object of the class, which is referred to as self, we can refer to its attributes by using a navigation expression, as in self.age in the class *Adult*. We can also refer to objects related to self by using a role name, as in self.father. We can therefore refer to attributes of related objects, for example, self.father.age.

SAQ 13

(a) In a model that contains the classes *Adult* and *Child*, where each *Child* has a mother and father that are instances of *Adult*, what constraints might you impose on relationships between the classes? Express the constraints in English.

(b) In the UK, it is a legal requirement that both parties to a marriage are at least 16. Should this be modelled as an invariant?

(c) How would you model the constraint in a hotel system that every bill must be paid with either a cheque or a credit card? How would you extend your model if cash were to be allowed?

ANSWER

(a) Every *Child* must have a mother and father that are instances of *Adult*. The father cannot be the mother. A father must be male, and a mother must be female. Both adults must be older than the child.

(b) Almost certainly not. A model is meant to express what the case *is* about, rather than what it *ought* to be about. So unless the domain expert agrees that illegal marriages will not need to be represented, our model should allow them.

(c) Figure 47 shows one way is to use the {*xor*} notation. However, this can relate only two associations. If you need to express a three-way constraint in order to allow for the addition of cash payments, for example, you will have to abandon the graphical notation of {*xor*}, and resort to writing a textual constraint. Alternatively, you could use generalisation to create an abstract *Payment* class with an association to the class *Bill*, following the examples in Section 4 of this unit. Then each payment method, such as cheque or credit card or cash, would become a specialisation of the parent, abstract class *Payment*.

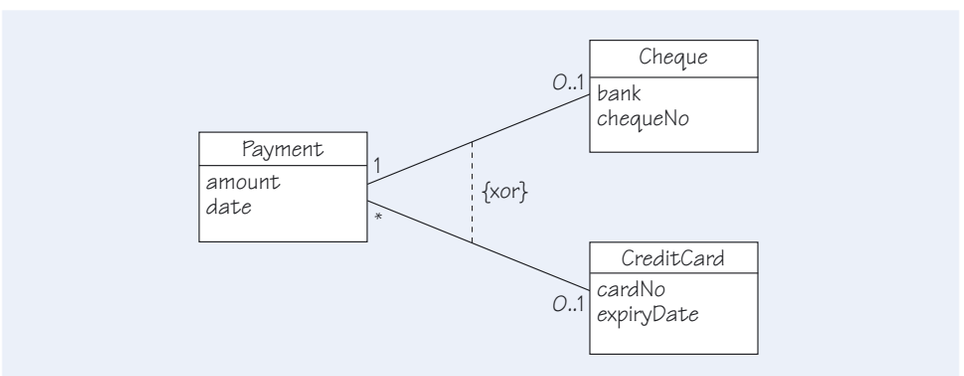

Figure 47 Payments, cheques and credit cards

5.3 Finding invariants by considering loops in associations

You have seen several examples in which a plausible-looking initial model turned out to be too general. The model was under-constrained, and needed an invariant to constrain it to allow only valid configurations. It can be extremely easy to overlook the need for an invariant. We all tend to interpret a model in our own ways, and remain oblivious to other interpretations. When the model is passed to someone else to implement, that person may not share our assumptions.

A particular type of invariant can arise from loops formed by associations. If there is a loop, there are two paths for getting from one class to another. Are these paths really independent, or is there actually a restriction that we should express? Look at Figure 48, which has three loops: there are two loops linking *Hotel*, *Guest* and *Room*, and one linking *Guest* and *Room*.

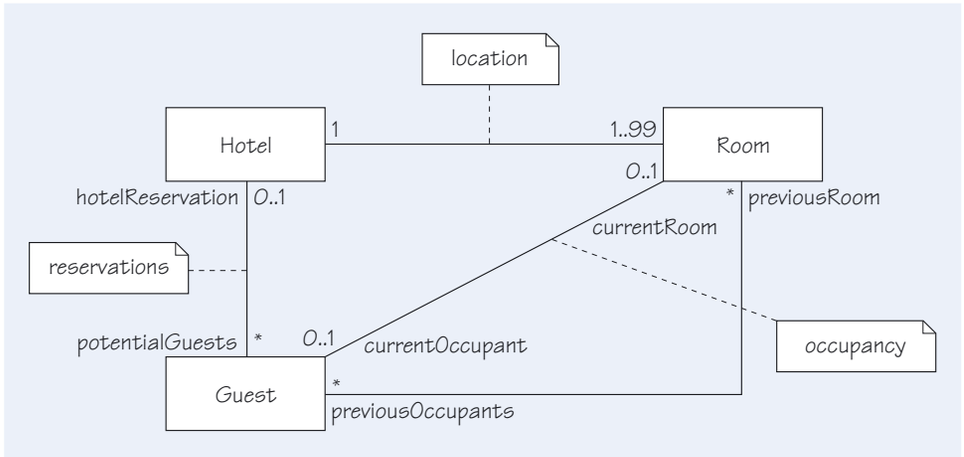

Figure 48 Examples of multiplicities

In this model, the hotel owns rooms, and guests occupy rooms. But a guest can have no more than one reservation at a time. There are three ways of starting at a guest and reaching a hotel. You can go directly from a guest to a hotel, or you can go via a room (and there are two ways of doing this). You can ask which hotel a guest has a reservation in, or which hotel owns the room that the guest is occupying. These routes must yield the same hotel. A guest cannot stay in a room belonging to *theRitz*, while at the same time having a reservation at *theSavoy*. Once you have spotted the need for a constraint, it does not matter how you record it. English is satisfactory, but you could write an invariant for the class *Guest* in OCL as follows:

```
context Guest inv:
    self.currentRoom.notEmpty() implies self.currentRoom.hotel = self.hotelReservation
```

Note that, as mentioned in the discussion of navigation expressions in Subsection 2.10, the default role name for an association end is the name of the class at that end of the association, but beginning with a lower-case letter. From Figure 48, for example, we have derived the default role name *hotel* at the *Hotel* end of the association between *Room* and *Hotel*.

One of the simplest forms of loop is when there are two or more associations between a pair of classes, or when a class is related to itself. There may well prove to be a constraint between the related objects. Figures 49 and 50 show some examples.

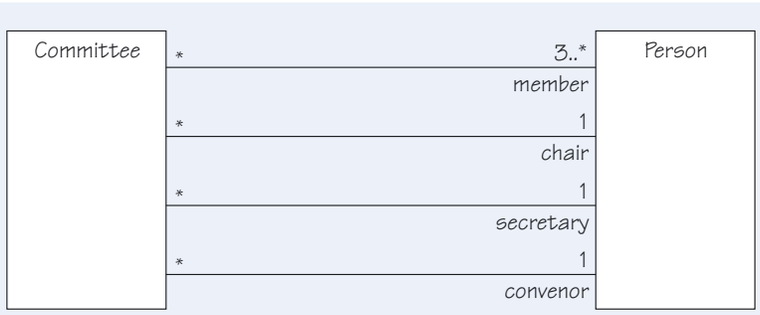

Figure 49 Some loops in a committee model

In Figure 49, the person who chairs a committee cannot be the same person as the secretary, although both must be members of the committee. However, the chair might also be the convenor. Some loops or cycles in a model may be harmless, in which case there will be no need to add a constraint. The loop involving the chair and the secretary, however, reveals that the model allows illegal situations, so you must constrain the model accordingly by, for example, placing the following constraint in the class *Committee*:

context Committee **inv**:
    self.chair <> self.secretary

The symbol <> should be read as 'not equals'.

In Figure 20, you saw that a library member could be associated with past loans and current loans. Figure 50 shows the same situation – this time with an intermediate class called *Loan*. Presumably no loan can be both a past and current loan simultaneously. In other words, the sets of loans associated with a member do not overlap. You can express this constraint in terms of non-overlapping sets using the {*xor*} notation: exactly one of the associations can be in use at any given time. Using {*xor*} shows that the same instances of *Loan* and *LibraryMember* cannot participate in the two associations at the same time. A loan is either current or past due.

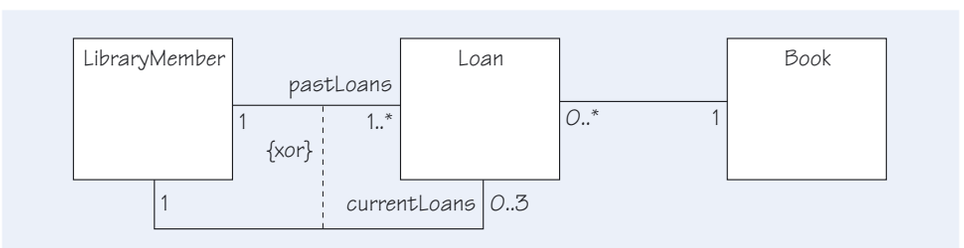

Figure 50 A possible loop in a class model for a lending library

This model illustrates a solution to two interesting issues. The first is how to represent the case where a member has no books currently on loan or any past due loans – the situation in which new members find themselves. Remember the meaning of {*xor*}: one and only one of the associations is active. It must therefore be the case that every member is linked either by the association with the *pastLoans* role (with multiplicity `1..*`) or by the association with the *currentLoans* role (with multiplicity `0..3`), but not both. A new member is obviously not linked to any past loans, so the association with the *currentLoans* role must be active. Since its multiplicity is `0..3`, the new member can be linked to zero loans as required. Note the subtle but important part played by the {*xor*} constraint.

The other issue is that if a loan exists, it must be linked to exactly one instance of *LibraryMember*. The model expresses this by using a multiplicity of `1` on both associations leading from *Loan* to *LibraryMember*. Again, remember that {*xor*} means that one or other but not both of the associations must be active; so either the association with the *pastLoans* role or the association with the *currentLoans* role is active. Either way, the *Loan* is linked to precisely one *LibraryMember*.

We have used the equality operator (=) without discussing what it means. You might assume that the equality operator in

context Guest **inv**:
    self.currentRoom.notEmpty() **implies** self.currentRoom.hotel = self.hotelReservation

is comparing equality of two objects. This is not necessarily the case, as a navigation expression does not always return a single instance of an object, but instead may return a set of objects. In addition, the object identity is being compared, not the object value. For example, using the model shown in Figure 48, the expression aHotel.room will return a set of instances of *Room* objects, (in theory) one for each room in the hotel.

For an equality expression to be true, the navigation expressions on each side of the operator must return the same set of instances. The sets are not equal if different instances are returned, even if the instances contain the same room information.

The inequality operator (<>) tests whether two sets are *not* identical, that is, that there is at least one object that is in one of the sets but not the other. Again, note that what is compared is instance identity, not value.

The includes operator (as in aHotel.room includes aGuest.currentRoom) tests whether the set returned by the expression on the right-hand side is a subset of the set returned by the left-hand side expression, that is, every instance in the right-hand-side set is also in the left-hand-side set.

SAQ 14

(a) The model in Figure 51 expresses the fact that a train must be associated with two employees: a driver and a guard. By considering the loop of associations, identify a problem with the model, and suggest a constraint that solves the problem.

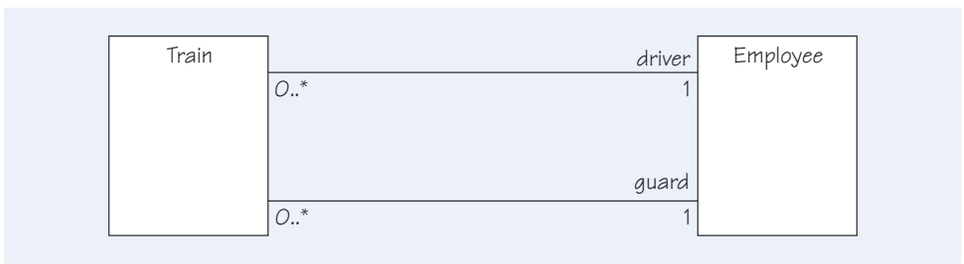

Figure 51 A loop that requires a constraint

(b) The model in Figure 52 shows that a person takes out a mortgage to buy a house. A person can take out one or more mortgages. But a person must offer that house as security against a given mortgage. By considering the loop of associations, identify a problem with the model, and suggest a suitable constraint using OCL.

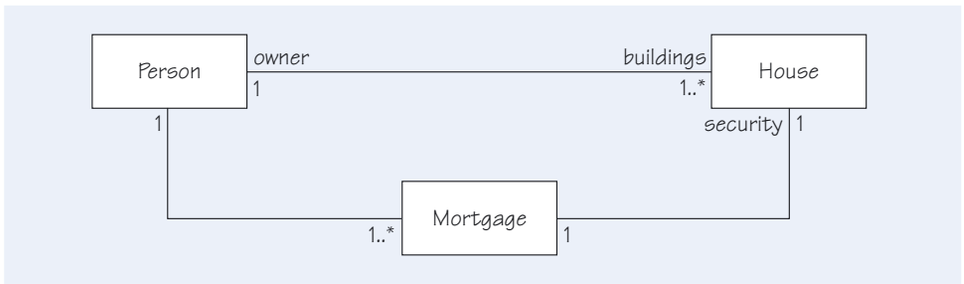

Figure 52 A loop that has a complex constraint

ANSWER

(a) You need to capture the fact that a given employee cannot be both the driver and the guard for a particular train. You can formulate this as a constraint on any class round the loop. For example, the following constraint can be located in the class *Train*:

```
context Train inv:
    self.driver <> self.guard
```

(b) The model would allow one person to have a mortgage that uses someone else's home as security. You need to capture the fact that a house's owner is the same person who offered that house as security for a given mortgage. When there are multiplicities in the loop that can be greater than one, you need to take care in writing the invariant. In one direction around the loop, an individual house has only one owner. In the opposite direction, each house is security for only one mortgage, and that mortgage is associated with only one owner. This makes it easy to place an appropriate constraint on either the *House* class or the *Mortgage* class. For the *House* class, we could write:

context House **inv**:
self.owner = self.mortgage.person

Similarly, for the *Mortgage* class, we could write:

context House **inv**:
self.person = self.security.owner

However, expressing an equivalent constraint on the class *Person* is more problematic because you would have to find the intersection of two sets: the dwellings owned by a particular person and the mortgages taken out by the same person. We suggest that you use English to formulate a suitable constraint in such circumstances. You could place the constraint inside a note and attach it to the appropriate class.

Exercise 10

Suppose that there is a class *Person* in a model, and you have identified the attributes *title*, *firstName*, *lastName*, *sex*, *houseNumber*, *streetName*, *city* and *postcode*,

all of which are represented as strings. Write an appropriate invariant that will constrain the attribute values to exclude invalid values.

Solution

We might constrain some fields to have appropriate capitals and constrain others to contain only values from a fixed set as follows:

{*title* must be spaces, or one of 'Mr', 'Mrs' or 'Ms';

firstName and *lastName* are written in lower case, but upper case is applied either to the first character of a word or after a space;

sex must be 'M' or 'F' or an empty string;

houseNumber must contain only digits, forming a number greater than zero, optionally followed by a letter;

streetName and *city* are written in lower case, but upper case is applied either to the first character of a word or any character after punctuation or spaces;

postcode may contain upper-case letters and digits, and a single space}

The invariants are certainly all things that must remain true, but they may not be the whole truth. For example, the constraint given above is *necessary* for a postcode to be valid, but it is not *sufficient*: a string could conform to it but still not be a postcode that actually exists. However, the constraint *will* always be true, if the system is implemented correctly.

Unfortunately, the invariant for *title* is probably incorrect. There are other commonly approved titles such as 'Dr' and 'Sir'.

Exercise 11

Figure 53 shows a fragment of a class model for a chain of video libraries. It includes an attribute called *location* for the class *VideoLibrary* to show where each branch is located. The class *Member* has been given two attributes: *name* and *address*. Identify a problem with the loop of associations in this model. How could an object diagram help you identify the problem?

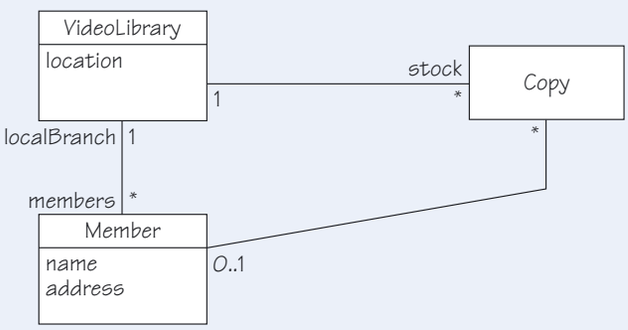

Figure 53 A class model for a video library

Solution

The loop in the associations for the video library's class model is similar to that for the hotel's class model that you saw in Figure 48. That is, there are two ways of starting with a member and ending with a library. You can ask the member either which library he or she is a member of, or which library owns the stock copy that the member is borrowing. Both routes must yield the same library.

In Section 2, you saw that an object diagram is very useful for determining whether a class model is correct and sufficiently constrained, because an object diagram represents a particular configuration at a particular moment in time. For example, you could show a particular valid configuration of video library objects as in Figure 54. In another object diagram, you might (try to) show an invalid configuration. If you can show an invalid configuration, then you might want to add constraints, or change the model, to prevent it.

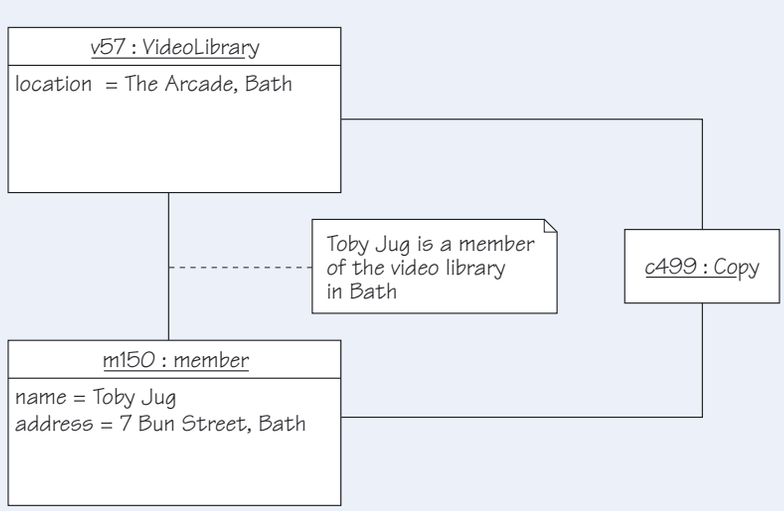

Figure 54 An object diagram for the video library

5.4 Summary of section

In the early stages of analysis, class models may be too general because they are not constrained by the rules that govern the domain in question. In this section, you have seen that most models require constraints to limit them to valid configurations, and you have been introduced to various notations for expressing them.

- In UML, attributes of a class can be constrained using an invariant in order to express one of the rules of the domain.
- UML provides a number of ways of expressing constraints on associations. There are special notations such as {*xor*}, as well as the formality of the Object Constraint Language or the informality of natural language. When choosing which notation to use, you should consider the skills of those who will read the model.
- Associations in a class model can form loops. Such loops may be totally independent, but can often hide a restriction that should be expressed as a constraint.

6 Summary

This unit has introduced the parts of UML needed to build class models of some sophistication. You have seen some of the shortcomings of modelling using classes. From the basic notation for classes and associations, you have developed more precise notations such as qualified associations. We have shown the need for constraints on over-general models and a way of constraining a model to reduce dependencies among classes.

At this point, you should be able to describe the static structure of a set of classes and their associations for your potential software system. At the conceptual modelling stage, your focus will be the key domain abstractions, their properties and their relationships. As your focus shifts to the specification of a solution, you will begin to associate conceptual elements with software elements in your model.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- create a list of candidate classes from a requirements document;
- model the classes and their attributes using UML;
- identify relationships between classes;
- understand the difference between class and object models;
- identify conditions that will limit the potential behaviour of model elements;
- identify the main limitations of class models;
- model associations between classes;
- identify and represent the navigability of associations in a class model;
- express generalisation/specialisation relationships in UML;
- identify dangerous dependencies;
- recognise the need for constraints on class models;
- express constraints within a single class;
- express constraints on associations;
- test models with loop associations for the need for an invariant.

Index